【教学服务系统】

——信息管理子系统

设计说明书

组长：钟志昊

组员：楼宸豪、李安璟、徐浩然、陈博凡

日期：2025.5.11

版本：1.2

目录

1 文档介绍 **5**
1.1 编写目的 . . . 5
1.2 文档范围 . . . 5
1.3 读者对象 . . . 5
1.4 术语与缩写解释 . . . 6
1.5 参考资料 . . . 6

2 项目介绍 **7**
2.1 项目说明 . . . 7
2.2 项目背景 . . . 7
2.3 需求概述 . . . 7
2.3.1 总体需求 . . . 7
2.3.2 性能需求 . . . 9
2.3.3 安全性需求 . . . 9
2.3.4 可维护性需求 . . . 11
2.4 条件与限制 . . . 13

3 总体设计 **14**
3.1 基本设计概念和流程处理 . . . 14
3.2 功能 IPO 图 . . . 15
3.2.1 登录账户 . . . 15
3.2.2 注册账户 . . . 15
3.2.3 管理员增删改账户信息 . . . 15
3.2.4 查询个人信息 . . . 16
3.2.5 修改个人信息 . . . 16
3.2.6 教师增删改课程信息 . . . 16
3.2.7 查询课程信息 . . . 17
3.2.8 教师上传成绩 . . . 17
3.2.9 教师/学生查询成绩 . . . 17
3.2.10 修改成绩 . . . 18
3.3 系统结构 . . . 18
3.4 技术介绍 . . . 23
3.4.1 前端框架 . . . 23
3.4.2 后端框架 . . . 23
3.4.3 数据库 . . . 23
3.4.4 ORM 框架 . . . 24
3.4.5 单元测试框架 . . . 24
3.4.6 认证与鉴权 . . . 24
3.5 部署图 . . . 24
3.6 架构语境图 . . . 25
3.7 类图 . . . 26

3.8 接口设计 . . . 26
3.8.1 内部接口 . . . 26
3.8.2 外部接口 . . . 28

4 详细设计 31
4.1 顺序图 . . . 31
4.1.1 登陆账户 . . . 31
4.1.2 注册账户 . . . 31
4.1.3 管理员增删改账户信息 . . . 32
4.1.4 查询个人信息 . . . 32
4.1.5 修改个人信息 . . . 33
4.1.6 教师增删改课程信息 . . . 33
4.1.7 查询课程信息 . . . 34
4.1.8 教师上传成绩 . . . 34
4.1.9 教师/学生查询成绩 . . . 35
4.1.10 修改成绩 . . . 35
4.2 执行概念 . . . 36
4.2.1 登录账户 . . . 36
4.2.2 注册账户 . . . 38
4.2.3 管理员增删改账户信息 . . . 39
4.2.4 查询个人信息 . . . 42
4.2.5 修改个人信息 . . . 44
4.2.6 教师增删改课程信息 . . . 47
4.2.7 查询课程信息 . . . 51
4.2.8 教师上传成绩 . . . 54
4.2.9 教师/学生查询成绩 . . . 56
4.2.10 修改成绩 . . . 60

5 用户界面 64
5.1 用户登录界面 . . . 64
5.2 系统主页界面 . . . 65
5.3 用户列表界面 . . . 65
5.4 课程列表界面 . . . 66
5.5 课程目录界面 . . . 66
5.6 成绩查询界面 . . . 67
5.7 成绩分析界面 . . . 68

6 数据库设计 70
6.1 概念结构设计 . . . 70
6.1.1 用户信息 . . . 70
6.1.2 课程信息 . . . 70
6.1.3 选修信息 . . . 70
6.1.4 班级信息 . . . 70
6.1.5 课表信息 . . . 71

6.1.6 总 E-R 图 72
6.2 逻辑结构设计 73
6.3 物理结构设计 73

7 运行设计 **76**
7.1 运行模块组合 76
7.2 运行控制 76
7.2.1 用户模块 76
7.2.2 管理员模块 77

8 系统出错设计 **77**
8.1 出错信息 77
8.2 补救措施 77
8.2.1 数据恢复机制 77
8.2.2 故障转移策略 77
8.2.3 日志审计 78
8.3 系统维护设计 78

1 文档介绍

1.1 编写目的

本文档描述软件产品功能设计说明书（SRS）的目的是：

1. 定义软件总体要求，作为用户和软件开发人员之间相互了解的基础；
2. 提供性能要求、初步设计和用户影响的信息，作为软件人员进行软件结构设计和编码的基础；
3. 作为软件总体测试的依据；
4. 为软件的后续维护和升级提供清晰的参考；
5. 为用户验收系统是否符合预期需求提供判定依据；
6. 为需求变更管理提供可追溯的文档支撑。

1.2 文档范围

教学服务系统–信息管理子系统软件产品设计说明书主要是对该系统中功能及功能操作方式进行基本的描述，包含了该模块的总体设计、详细设计、界面设计、数据库设计、运行设计的详细介绍，包括功能 IPO 图、部署图、系统结构、类图、顺序图、运行模块组合等内容。

1.3 读者对象

本软件需求说明书的预期读者包括：

1. 客户
2. 系统开发人员
3. 测试人员
4. 需求分析人员
5. 项目管理人员
6. 系统维护人员

1.4 术语与缩写解释

缩写及术语	解释
SOA 架构	面向服务的体系结构。
元数据	描述数据的内容、格式、来源和其他有关特性的数据信息。
Metadata	同“元数据”。
数据中心	以分类数据为核心，依托成熟的存储、数据库、GIS 等技术。
Data Center	网络等技术，按照统一标准，建立的具有信息管理、分析、查询、统计及服务的一体化数据管理体系。
数据管理（Data Management）	利用数据库、数据仓库、元数据和数据交换技术，进行采集、集中和共享的方法和技术的集合。包括数据模型设计、数据集成、数据质量管理与数据安全等，是实现数据治理的基础。
顺序图（Sequence Diagram）	表示对象之间的交互，强调消息传递的时间顺序。纵向代表时间推进，横向表示对象，适用于系统的动态行为建模。
部署图（Deployment Diagram）	展示系统的物理架构，描述软件模块部署在哪些硬件节点上及其之间的连接，适用于系统部署设计与分析。
功能 IPO 图	采用 IPO 模型（Input/Process/Output），描述系统或模块输入、处理过程与输出之间的关系，常用于功能分析与系统设计初期。
类图（Class Diagram）	表示类的静态结构，包括属性、方法以及类与类之间的关系（如继承、关联、依赖等），是面向对象建模的基础图之一。

表 1: 术语解释汇总表

1.5 参考资料

序号	文档名称	作者	版本	发布日期
1	软件工程: 实践者的研究方法	[美] Roger S. Pressman, [美] Bruce R. Maxim 著	9	/

2 项目介绍

2.1 项目说明

项目名称：教学服务系统–信息管理子系统
任务提出者：软件工程基础任课老师-张引
开发者：浙江大学 2024-2025 学年夏学期软件工程基础课程 B 大组第一小组
用户群：各个学校教师、学生和教务管理人员等

2.2 项目背景

浙江大学软件工程基础课程在内容上分为理论课和实验课两大部分。理论课由老师介绍软件工程中的相关理论知识，实验课采取分组完成形式，每 5 人组成 1 小组，每 5 小组组成 1 大组，合作进行软件系统开发。本学期课程中老师选择教学服务系统作为综合性实验题目。

教学服务平台是面向高校的一站式数字化教学管理系统，旨在优化教学流程、提升管理效率、增强师生互动。该系统采用模块化设计，涵盖信息管理、课程安排、选课管理、在线测验、资源共享五大核心功能模块，为教师、学生和管理员提供便捷的在线服务。其中，信息管理子系统为系统的核心支撑模块，负责用户账户、权限控制、课程信息与成绩信息的统一管理。本组主要负责信息管理子系统的开发。

在高校内部开发的教学管理系统中，往往注重教务操作而忽略了师生间的交互体验，缺乏灵活的选课机制、智能化的排课方案、以及数据驱动的成绩分析功能。同时，部分系统在用户权限管理、系统安全性与并发性能方面也存在短板，难以支持大规模用户同时访问和操作。基于对当前主流教学服务系统的深入调研与分析，本项目提出构建一套功能完善、结构清晰、用户友好的教学服务系统，整合信息管理、课程安排、课程选择、在线测验与资源共享等核心模块，致力于为高校师生提供一体化、高效、可靠的教学信息化服务平台

2.3 需求概述

2.3.1 总体需求

表 2: 用户信息管理功能列表

序号	一级功能	功能编号	二级功能	功能编号
1	账户管理	uim01	添加账号	uim011
			删除账号	uim012
			搜索账号	uim013
			编辑账号	uim014
			设置用户类型和权限	uim015
2	个人信息修改	uim02	添加个人信息	uim021
			修改个人信息	uim022
			上传图片	uim023

表 3: 课程基本信息管理功能列表

序号	一级功能	功能编号	二级功能	功能编号
1	课程管理	cim01	增加课程	cim011
			删除课程	cim012
			修改课程	cim013
2	课程搜索	cim02	搜索课程	cim021

表 4: 成绩管理功能列表

序号	一级功能	功能编号	二级功能	功能编号
1	成绩查询	gm01	上传成绩	gm011
			查询成绩	gm012
2	成绩修改	gm02	提交修改申请	gm021
			审核修改申请	gm022
3	成绩分析	gm03	求解平均成绩	gm023
			绘制成绩分布图	gm031
			统计成绩排名	gm032

表 5: 用户权限管理模块功能列表

序号	一级功能	功能编号	二级功能	功能编号
1	账户管理	uam01	用户注册	uam011
			用户登录	uam012
2	角色管理	uam02	增加角色	uam021
			删除角色	uam022
			修改角色权限	uam023
3	访问控制	uam03	用户登出	uam031
			接口访问鉴权	uam032

表 6: 系统安全管理模块功能列表

序号	一级功能	功能编号	二级功能	功能编号
1	账户管理	ssm01	用户信息管理	ssm011
			密码管理	ssm012
2	入侵防范管理	ssm02	入侵防范与异常检测	ssm021
3	系统日志记录	ssm03	安全日志记录	ssm031
			操作日志记录	ssm032
			审计日志保护	ssm033
			日志检索	ssm034

2.3.2 性能需求

系统应具备良好的总体性能表现，以保证在实际使用过程中提供流畅、稳定的用户体验。具体要求如下：

1. **系统响应速度**：在网络环境良好的情况下：
   - 单个用户在线时，Web 界面响应用户操作的时间应小于 1 秒，信息搜索操作响应时间应小于 2 秒。
   - 当 500 个用户同时在线时，Web 界面响应时间应小于 2 秒，信息搜索操作响应时间应小于 5 秒。
   - 系统应在高并发条件下保持响应稳定，无明显卡顿、无长时间无响应现象。
2. **并发访问能力**：系统应支持至少 500 个用户的同时在线访问，保证在用户高峰期依然能正常使用各项功能。此外，应根据实际访问量预留一定扩展空间，支持后续通过服务器扩展、负载均衡等手段提升并发能力。
3. **服务器配置要求**：最低服务器硬件配置要求为：CPU 4 核 2.6GHz 及以上，内存 8GB 及以上，硬盘 7200 转/分钟或以上。
4. **浏览器兼容性**：系统应兼容主流 Web 浏览器，包括但不限于 Firefox、Chrome、Edge，且在不同浏览器及常见分辨率下均应保持界面一致性与功能正常。此外，系统应适配 PC 端常规使用环境，确保响应式布局良好，支持常见分辨率（如 1920×1080、1366×768 等）。
5. **数据实时性与准确性**：系统应确保前端展示的数据能够实时同步后端数据库的最新信息，避免出现数据延迟、缓存过期等问题。所有用户查询、检索等操作返回的数据必须准确、完整，不得出现因同步问题导致的信息丢失或异常。

2.3.3 安全性需求

信息管理子系统是教学服务系统的基础，为各个模块提供登录认证和鉴权服务，教学服务平台涉及到大量教师和学生的个人隐私信息，对于安全性有较高的要求，为此我们提出以下几点要求：

1. **保密性**

(a) 本系统采用账号和密码认证机制，并预留与浙江大学统一身份认证平台对接的能力。用户访问系统需验证输入的账户和密码信息是否与数据库中存储的一致，以防止未授权用户访问系统资源，同时防止权限不足的用户执行管理员相关操作。

(b) 要求用户设置 8 位以上的复杂密码（包含大小写字母、数字及特殊字符），以提高密码强度，降低暴力破解的风险；系统定期提示用户更新密码，以防遗忘及提升账户安全性。

(c) 采用基于角色的访问控制（RBAC）模式，按最小权限原则分配系统权限，确保各类用户仅能访问和操作与其职责相关的资源，防止越权操作或滥用权限。

(d) 对于敏感数据（如密码、成绩等），在前后端数据传输过程中应采用加密传输（如 HTTPS）；密码在数据库中应以安全哈希算法（如 bcrypt、scrypt）加盐存储，禁止明文保存，以提升数据安全性。

(e) 用户登录过程中需防范 SQL 注入攻击、暴力破解攻击等常见安全威胁；系统应设置登录失败次数限制，并在连续多次登录失败后采取措施（如临时封禁或增加验证码）；此外，对于长时间无操作的登录会话，应自动强制登出，以防止账户被盗用。

2. **约束性**

(a) 对数据库的各类操作，特别是删除和修改设置严格约束，增加操作前确认机制如二次确认弹窗或多重验证，并在检测到批量或高风险操作时向用户发出显著警告提示，以有效防止恶意或误操作导致的数据丢失、损坏等问题。

(b) 对系统输入数据实施严格的校验规则，例如验证学号格式、成绩范围、用户名规范等，配合使用多重黑名单和白名单机制，确保过滤非法字符与异常请求，从源头防范 SQL 注入、脚本注入等安全风险。

(c) 对批量数据处理操作，如成绩导入、用户批量添加，设置合理阈值，如单次操作记录数 500 条，并在超限时进行分批处理提示，以防止带宽资源被瞬时占满，同时保证系统的稳定性和响应速度。

3. **完整性**

(a) 为重要数据表（如用户表、课程表、成绩表）设置完整性约束（如外键约束、唯一性约束、非空约束），防止插入非法或不符合规范的数据。

(b) 针对批量数据处理操作（如批量导入、批量更新）引入事务机制（Transaction），保证操作的原子性，即要么全部成功，要么全部失败，避免数据部分更新导致的不一致问题。

(c) 定期备份数据库，确保在遭受意外故障（如服务器宕机、攻击破坏）时可以通过备份快速恢复数据，最大程度减少数据丢失。

(d) 对关键数据操作（如添加、修改、删除）进行日志记录，包含操作者身份、操作时间、操作内容等信息，便于后续审计和追溯异常操作来源。

(e) 引入数据校验与异常检测机制（如校验逻辑、触发器、异常告警），在系统运行过程中实时检测潜在的数据完整性风险，并及时修正。

4. **用户信息安全**

(a) 着重保障系统本体安全，采取多层次防护措施防止外部未授权人员通过网络漏洞、系统弱点等途径侵入系统，确保系统边界安全。

(b) 系统设置会话超时自动注销机制：普通用户连续 15 分钟无操作将被强制登出，管理员账户连续 5 分钟无操作即强制登出，防止因用户离席、失误或设备被攻击者利用导致的账户信息泄露及越权风险。

(c) 对系统异常登录行为（如同一账户短时间内多地登录、连续多次登录失败）进行监控和报警，及时发现潜在攻击行为，保障系统运行环境的整体安全。

5. **防御攻击**

(a) 设置连续输入错误密码五次即锁定账户并中止认证，防止暴力破解攻击；锁定后需通过二次验证（如邮箱验证码、管理员解锁）才能重新启用账户。

(b) 后端系统统一采用 ORM（对象关系映射）框架进行数据库交互，自动过滤非法输入，降低 SQL 注入等注入式攻击的风险。

(c) 部署实时异常行为监控系统，针对高频访问、越权操作、异常登录、敏感数据批量导出等可疑行为进行识别，一旦检测到异常，立即通过邮件、短信、告警系统等方式通知系统管理员，并可根据规则自动采取应急措施（如封禁 IP、强制登出等）。

(d) 建立完善的漏洞响应标准作业程序（SOP），明确漏洞分级、修复流程与责任人，要求关键级别（高危）漏洞自发现起 24 小时内完成修复，并定期开展安全自检与渗透测试，提前排查潜在风险。

6. **安全审计功能**

系统需对所有敏感操作（如账户管理、权限调整、数据修改、系统配置变更等）进行完整记录，确保审计日志信息全面、准确。审计日志应详细记录操作者身份、操作时间、操作对象、操作内容及操作结果等关键信息，并要求日志保留时间不少于 2 年，同时采用防篡改措施（如哈希签名或专用隔离存储）以保障日志数据的完整性与可信性。

7. **数据加密**

系统应使用加密技术保障敏感数据在传输和存储过程中的机密性与完整性。在前后端数据交互过程中，应采用 HTTPS 协议替代 HTTP 协议，利用 SSL/TLS 协议栈对传输信息进行加密，防止未经授权的用户通过抓包等手段窃取明文数据。在敏感数据存储方面，尤其是用户密码，应在保存前进行加盐处理，即为每个密码添加随机前缀后再计算其哈希值，并仅存储哈希结果，避免因数据库泄露导致密码信息被直接破解，从而有效提升系统整体的数据安全性和抗攻击能力。

8. **访问控制**

系统采用基于角色的访问控制（RBAC）机制，将用户划分为管理员、教师和学生三种基本角色，各角色具备不同的权限范围。在各个子系统中，管理员拥有最高权限，有权访问和管理系统配置、用户管理等系统管理相关功能；教师拥有授课相关权限，可管理课程信息、录入学生成绩、发布学习资源等；学生拥有学习相关权限，可查看课程内容、参与学习活动、查询个人成绩等。通过细粒度的角色划分与权限管理，系统有效保障了资源访问的合理性与安全性，防止越权操作与信息泄露。

2.3.4 可维护性需求

作为一个成熟的系统，在开发初期就应该充分考虑系统的可维护性。对此，我们提出以下几点要求：

1. **模块划分**

   本项目在系统设计过程中，遵循高内聚、低耦合的原则进行模块划分。开发者需充分考虑模块内部结构的紧密性与模块间的独立性，确保各模块功能明确、职责单一，内部实现细节对外部屏蔽，模块之间通过清晰、标准化的接口进行通信。一方面，不同模块之间需合理划分功能边界，避免交叉依赖和功能重叠；另一方面，系统整体采用前后端分离的架构模式，前端主要作为用户交互界面，负责界面展示、用户操作监听与请求发送，所有核心业务逻辑、数据处理与权限控制等工作均在后端完成处理。

2. **文档规范**

   本项目将提供完备、清晰且具有良好可读性的文档资料，共包括以下四个部分：需求文档、设计文档、测试报告和系统使用说明书。每份文档均遵循统一的编写规范，采用规范化、准确的语言进行描述，确保内容条理清晰、逻辑严谨，便于项目各参与方阅读与理解。同时，文档中将结合流程图、结构图、时序图、界面示意图等多种图表形式，直观展示系统的结构、功能与工作流程，进一步提升文档的表达效果和可用性，为开发、测试、维护与用户使用等各个开发阶段提供有力支撑。

3. **代码规范**

   开发过程中需严格遵循所用编程语言的规范，系统主要开发语言为 C#，要求类名、接口名、枚举名等使用大驼峰命名法（PascalCase），方法名、属性名、事件名使用小驼峰命名法（camelCase）。整体代码注释覆盖率不得低于 30%，关键算法部分需补充时间复杂度与空间复杂度分析说明；同时，要求尽量减少重复代码，合理封装与抽象，提升代码的可读性、复用性与整体质量。

4. **测试保障**

   项目将建立完善的金字塔测试体系，覆盖单元测试、集成测试等多个层级，确保系统质量和稳定性。单元测试阶段要求：核心模块的测试覆盖率不低于 80%，非核心模块不低于 60%，并针对关键业务链路设计包括异常场景在内的测试用例，例如模拟网络中断、并发冲突、数据异常等情况，全面验证系统的稳定性与鲁棒性。在集成测试阶段，重点验证各模块之间的接口契合度与整体业务流程的正确性，确保系统功能协同运作无误。

5. **可扩展性**

   系统采用插件化架构设计，针对关键扩展点（如成绩计算规则等）预留标准化接口，支持插件的热插拔与独立升级，确保系统具备良好的扩展性和可维护性。数据库设计中预留约 20% 的备用字段，以便后续功能扩展或结构调整时灵活适配。

6. **自动化测试**

   系统构建完善的 CI/CD 流水线，支持代码提交后自动触发单元测试与静态代码扫描，保障代码质量与规范性。同时，针对核心业务功能实现自动化回归测试，确保关键功能在持续迭代中的稳定性。每次测试执行后生成标准化测试报告，报告中包含测试覆盖率、缺陷检测结果以及基于代码变更范围的影响分析，便于快速评估修改对系统整体的影响程度，提升交付效率与系统可靠性。

7. **版本控制策略**

   项目采用标准的 Git Flow 工作流进行代码管理，确保开发、测试与发布流程规范有序。版本号管理遵循语义化版本控制（Semantic Versioning，SemVer）规范，格式为主版本号. 次版本

号. 修订号，明确区分功能新增、向后兼容性更新与破坏性变更。每次版本发布时，自动生成并维护变更日志（CHANGELOG.md），详细记录新增特性、修复内容及已知问题，并对破坏性变更进行显著标注，确保开发团队和用户能够准确了解版本演进情况。

2.4 条件与限制

本项目部署在校内自建服务器上，保证服务器以及客户端之间通信畅通。

- 服务器：
  1. 类型：校内自建服务器
  2. CPU：Intel Xeon E5-2666 V3，10 核 20 线程
  3. 内存：32GB RAM
  4. 存储：2TB 机械硬盘（HDD）
  5. 操作系统：Ubuntu Server 22.04 LTS
- 通讯设备：
  1. 网线：正常联通且数据传输能力好
  2. 网卡：100M
- 客户端外围设备：
  1. 键盘：可正常使用
  2. 鼠标：可正常使用
  3. 显示器：全彩显示，可正常使用
  4. 硬盘：大于 100GB，转速大于 7200rpm
- 软件依赖：
  1. 操作系统：Windows10/11,Max OS, Linux
  2. 数据库平台：PostgreSQL
  3. 前端开发语言：html + css +C#
  4. 前端开发框架：Blazor
  5. 后端开发语言：C#
  6. 后端开发框架：ASP.NET.Core 9.0
  7. ORM 框架：Entity Framework Core
  8. 单元测试框架：xUnit
  9. 前端组件库：Ant Design Blazor
  10. 认证/鉴权方案：Json Web Token(JWT)
  11. 开发工具：Visual Studio 或者 Visual Studio Code
  12. 管理软件：DataGrip + PgAdmin4 + Docker
  13. 运行时环境：.NET Runtime 9.0
  14. 建模工具：Microsoft Word，draw.io 等
  15. 浏览器：Chrome、Edge、FireFox

3 总体设计

3.1 基本设计概念和流程处理

本子系统是一个横跨前后端的大项目，主要负责账户登录/注册、个人信息管理、课程管理、课程查询、成绩管理、成绩查询、管理员管理账户权限等。

前端：Blazor 框架

后端：ASP.NET.Core 框架

数据库：PostgreSQL，使用 Entity Frame Core 作为 ORM 框架

客户端：考虑到本系统所使用的框架较新，推荐使用 Chrome、Firefox、Safari 或 Microsoft Edge 浏览器。不推荐使用任何一个版本的 IE 浏览器，如需使用，版本须在 11 及以上。

处理流程图如下：

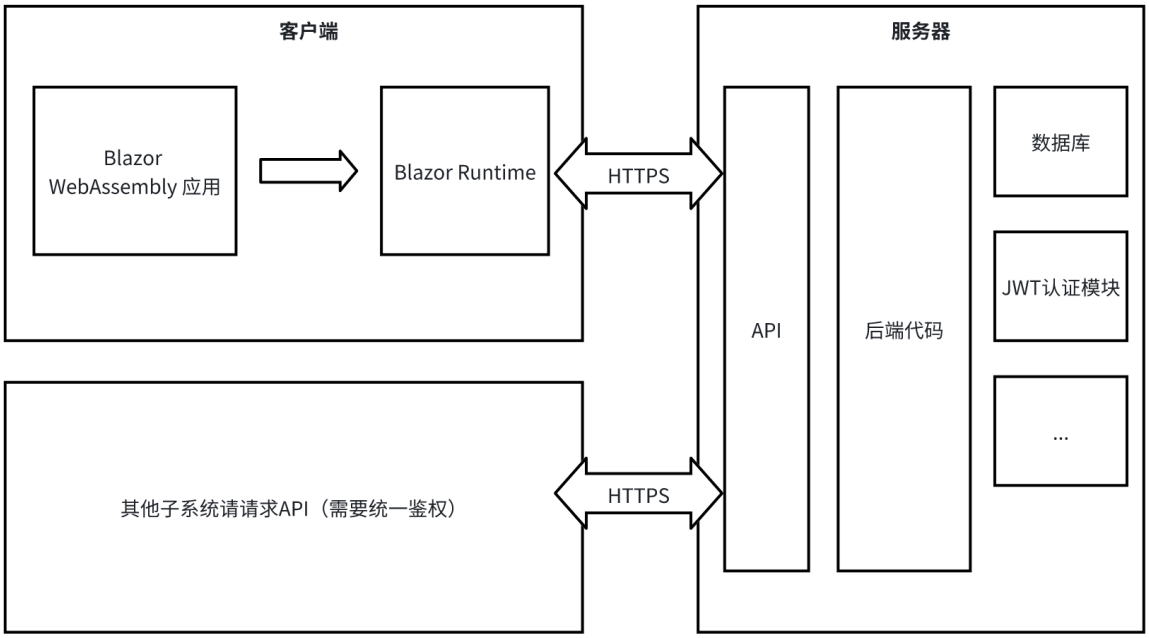


图 1: 处理流程图

3.2 功能 IPO 图

3.2.1 登录账户

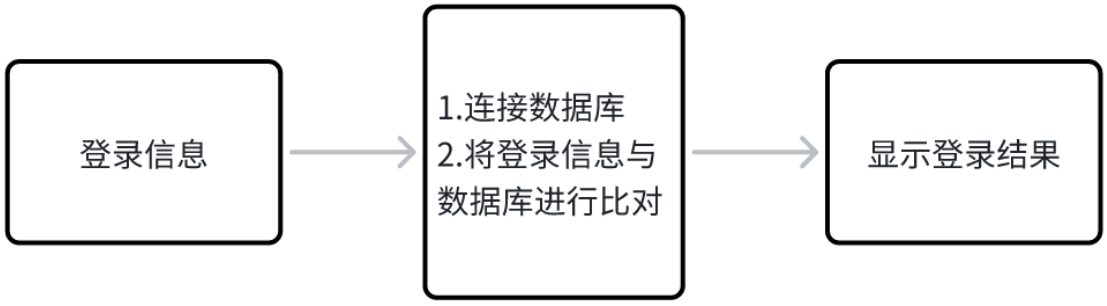


图 2: 用户登录 IPO 图

3.2.2 注册账户

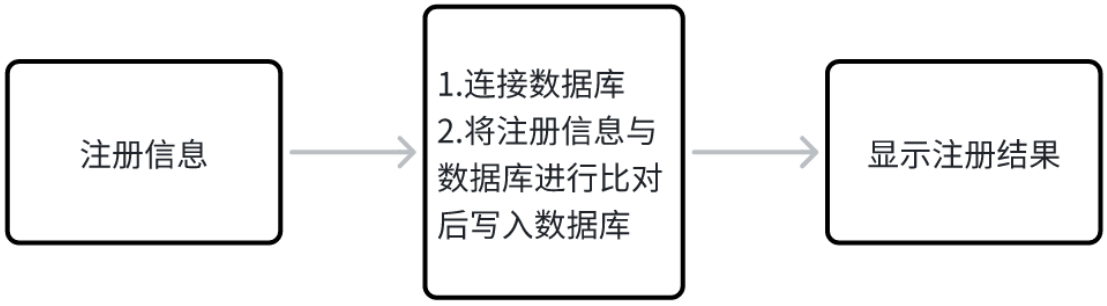


图 3: 用户注册 IPO 图

3.2.3 管理员增删改账户信息

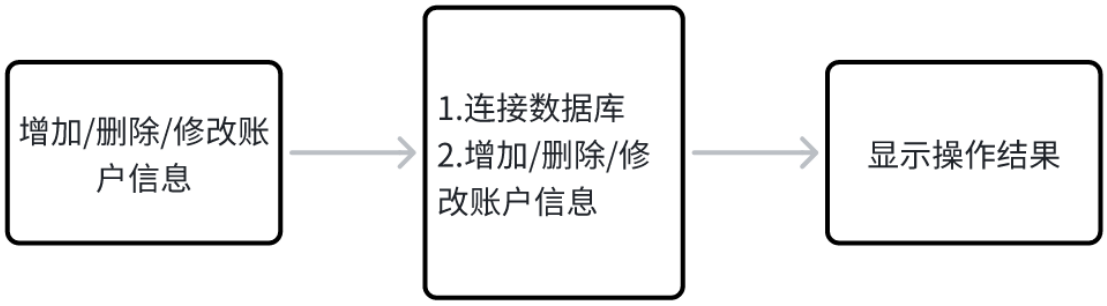


图 4: 管理员增删改账户信息 IPO 图

3.2.4 查询个人信息

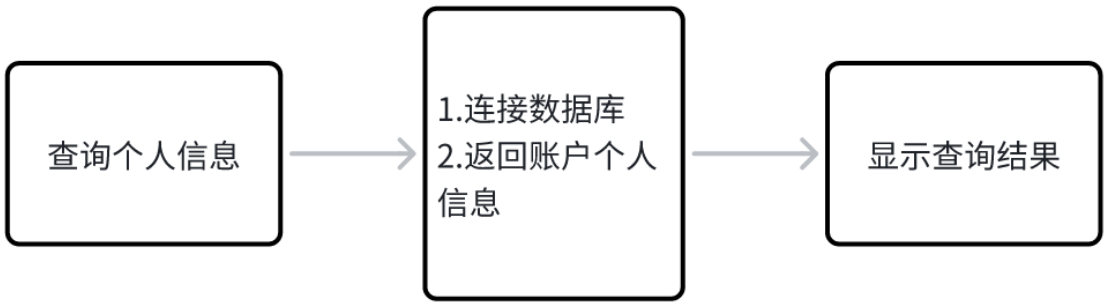


图 5: 查询个人信息 IPO 图

3.2.5 修改个人信息

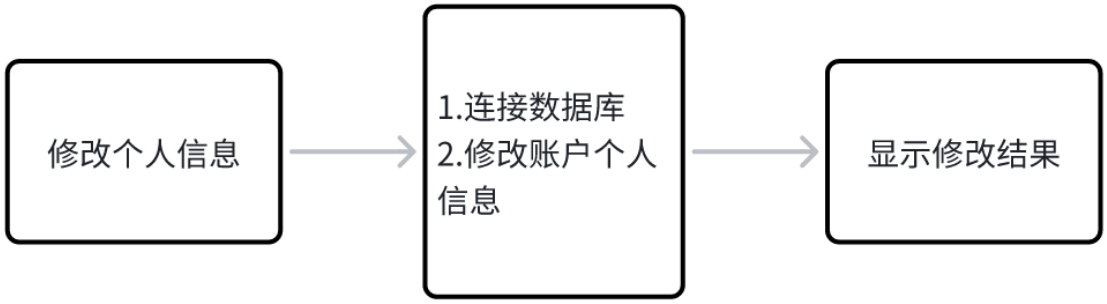


图 6: 修改个人信息 IPO 图

3.2.6 教师增删改课程信息

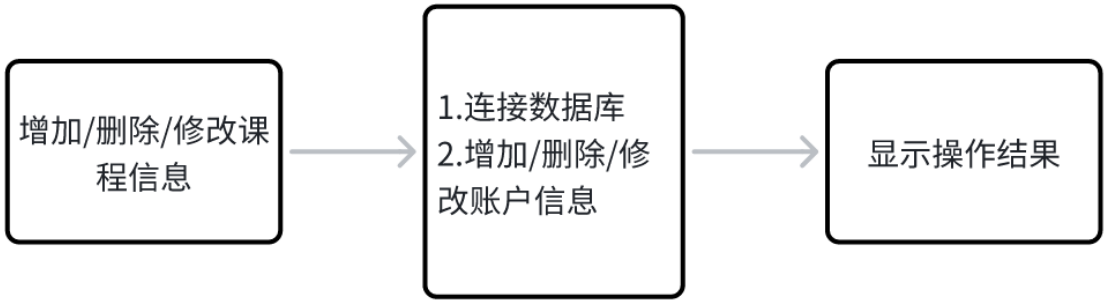


图 7: 教师增删改课程信息

3.2.7 查询课程信息

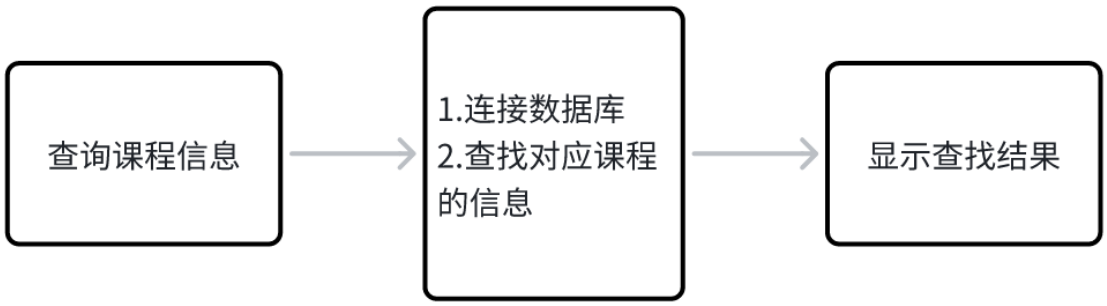


图 8: 查询课程信息

3.2.8 教师上传成绩

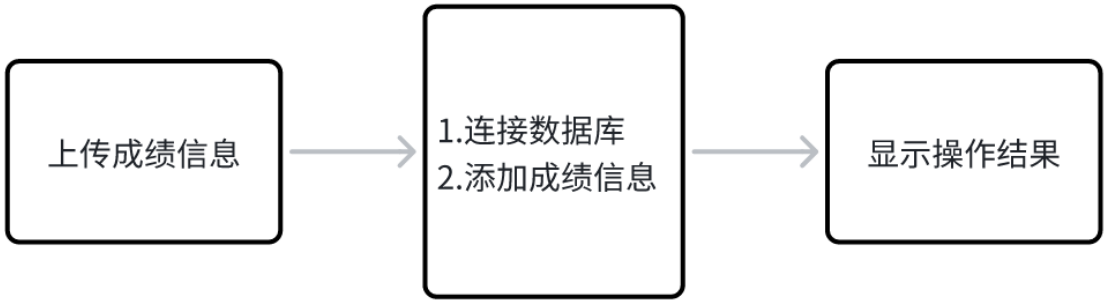


图 9: 教师上传成绩 IPO 图

3.2.9 教师/学生查询成绩

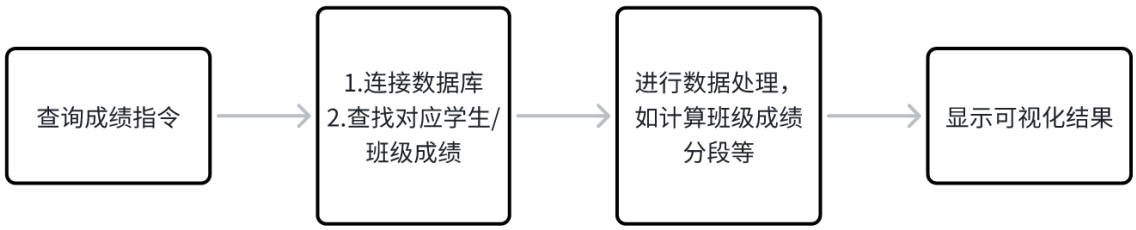


图 10: 教师/学生查询成绩

3.2.10 修改成绩

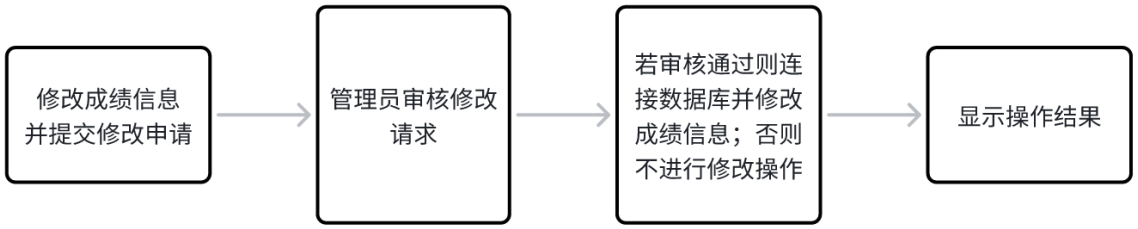


图 11: 修改成绩

3.3 系统结构

在需求分析中，我们得出的用例图如下，因为我们打算采用基于角色的访问控制，但是有部分功能是所有角色共有的，箭头交叉后可能较为复杂，所以除了汇总的用例图外，为每种角色绘制了单独的用例图：

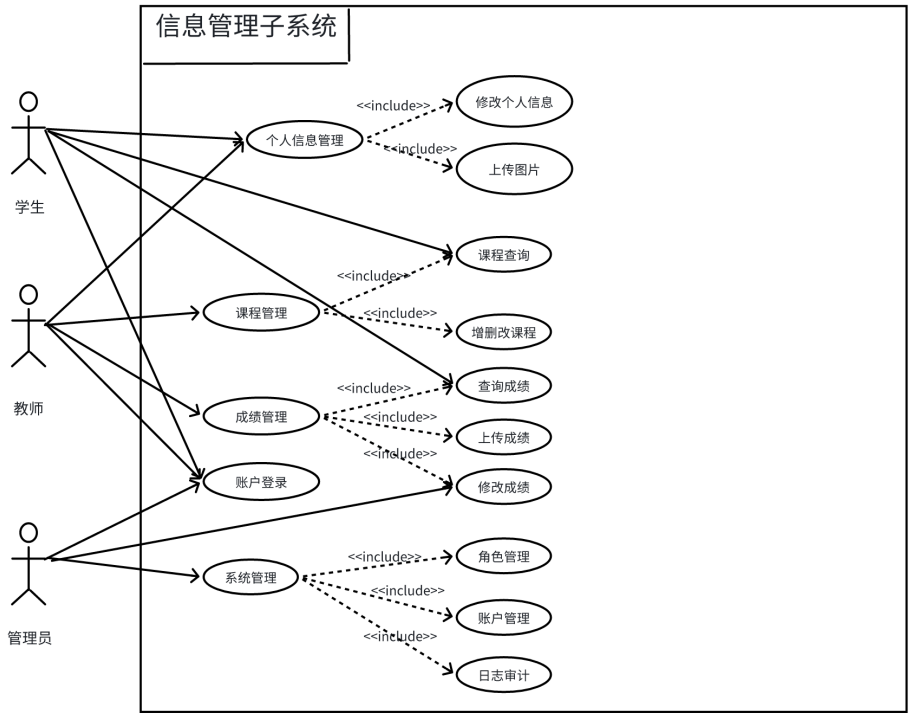


图 12: 总用例图

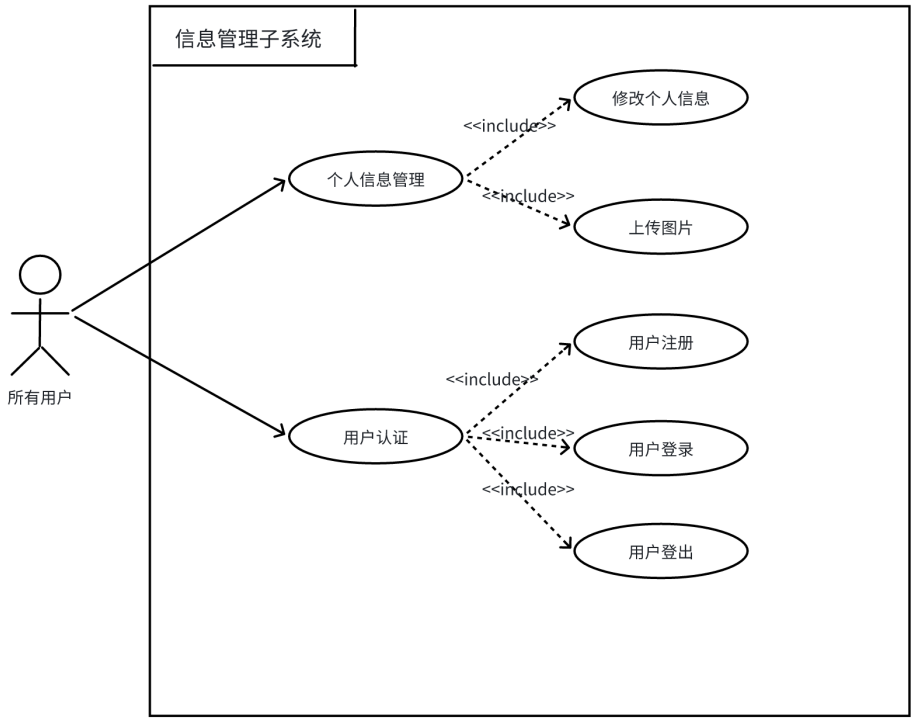


图 13: 所有用户用例图

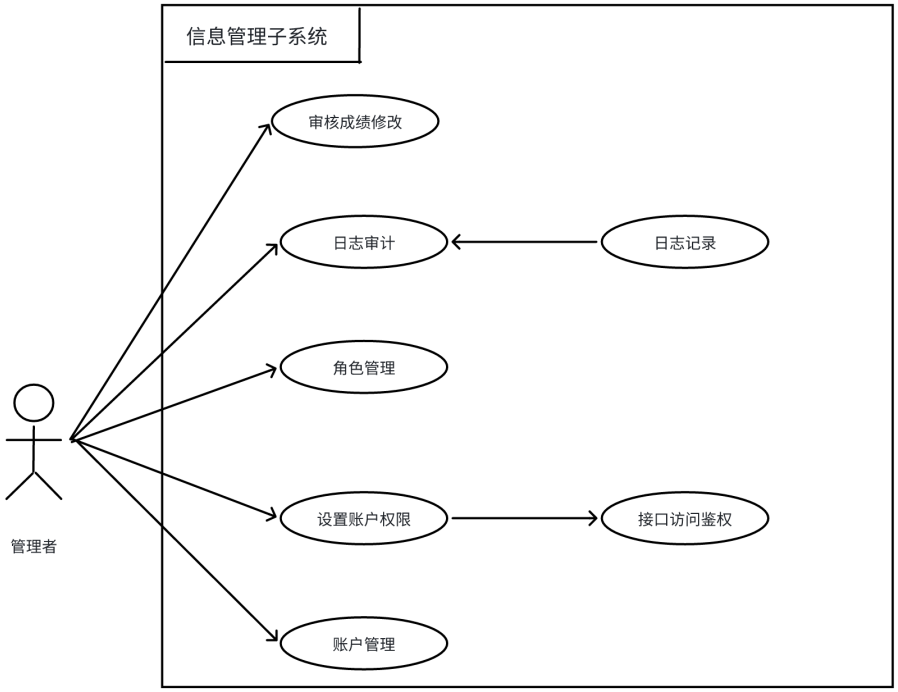


图 14: 管理员用例图

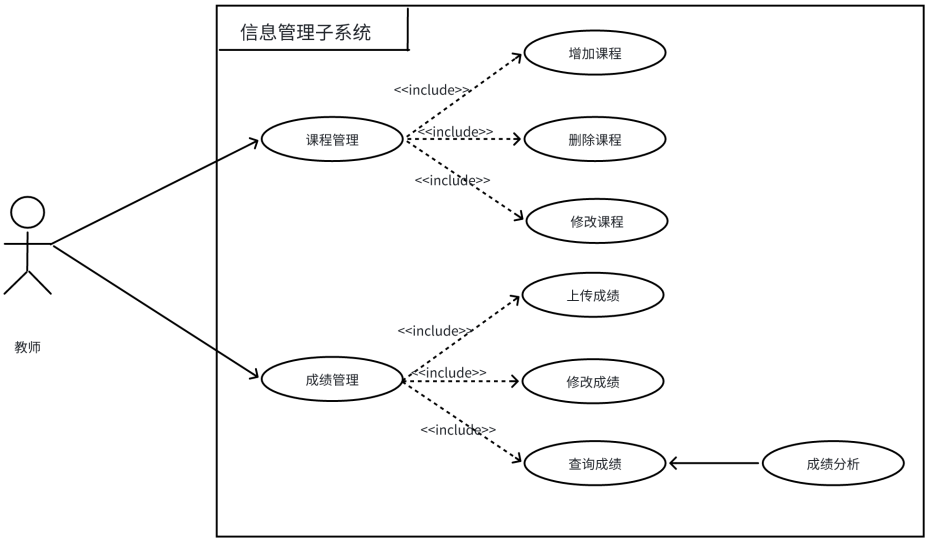


图 15: 教师用例图

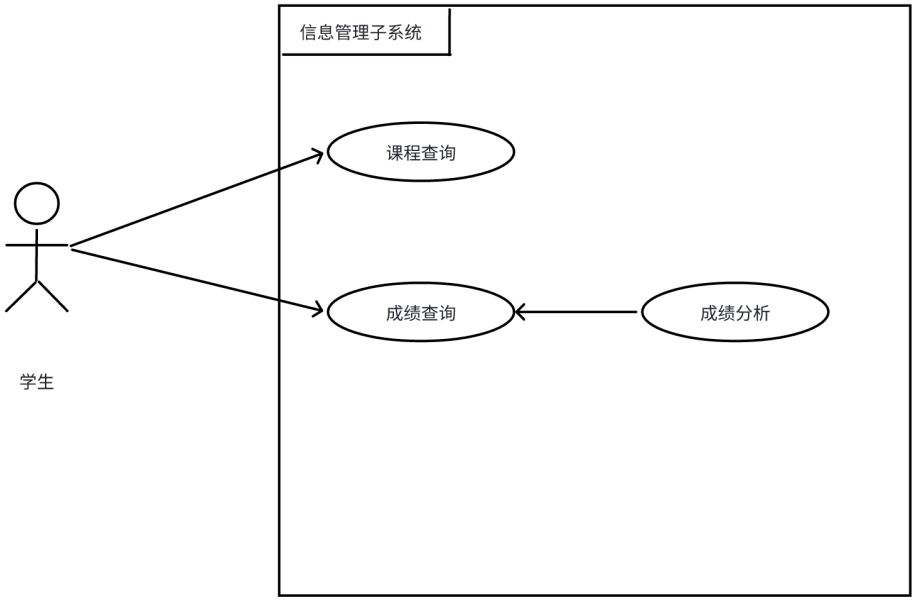


图 16: 学生用例图

顶层图如下，表现出系统的数据输入、输出过程：

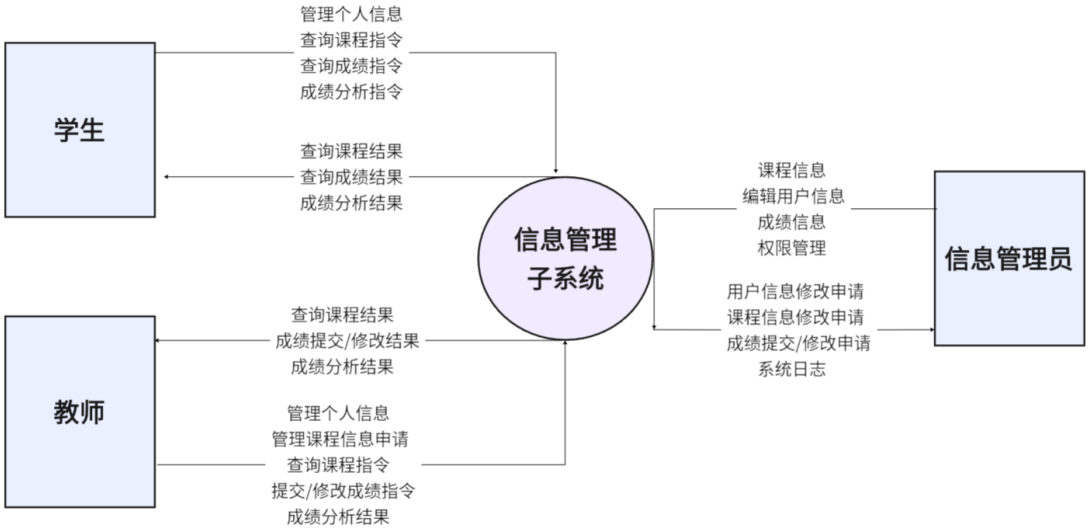


图 17: 顶层图

0 层图如下，对系统模块进行细化分类：

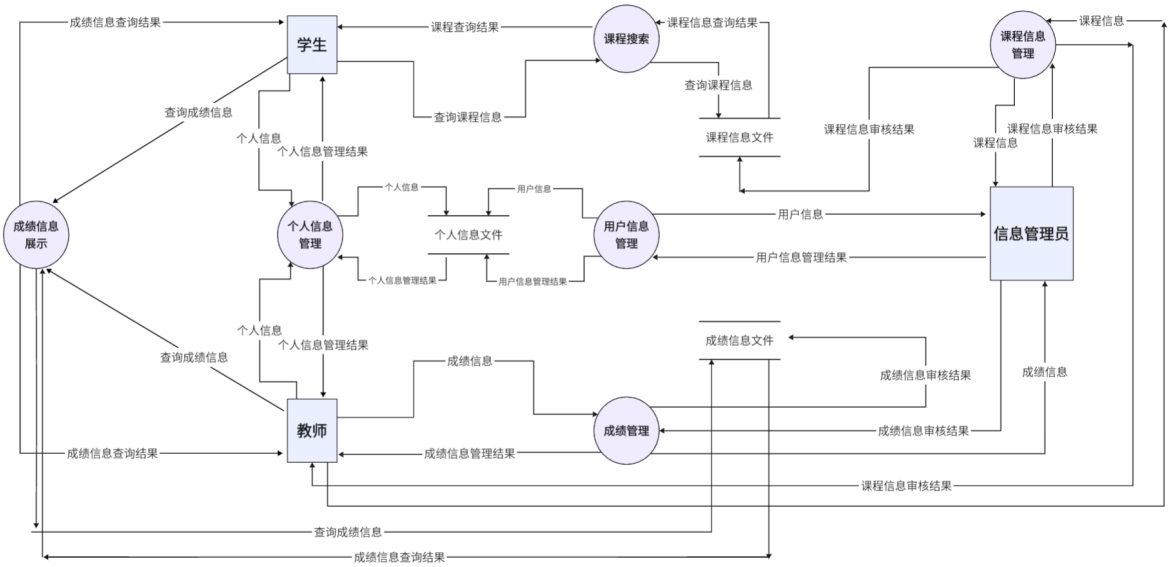


图 18: 0 层图

系统的整体结构如下图所示：

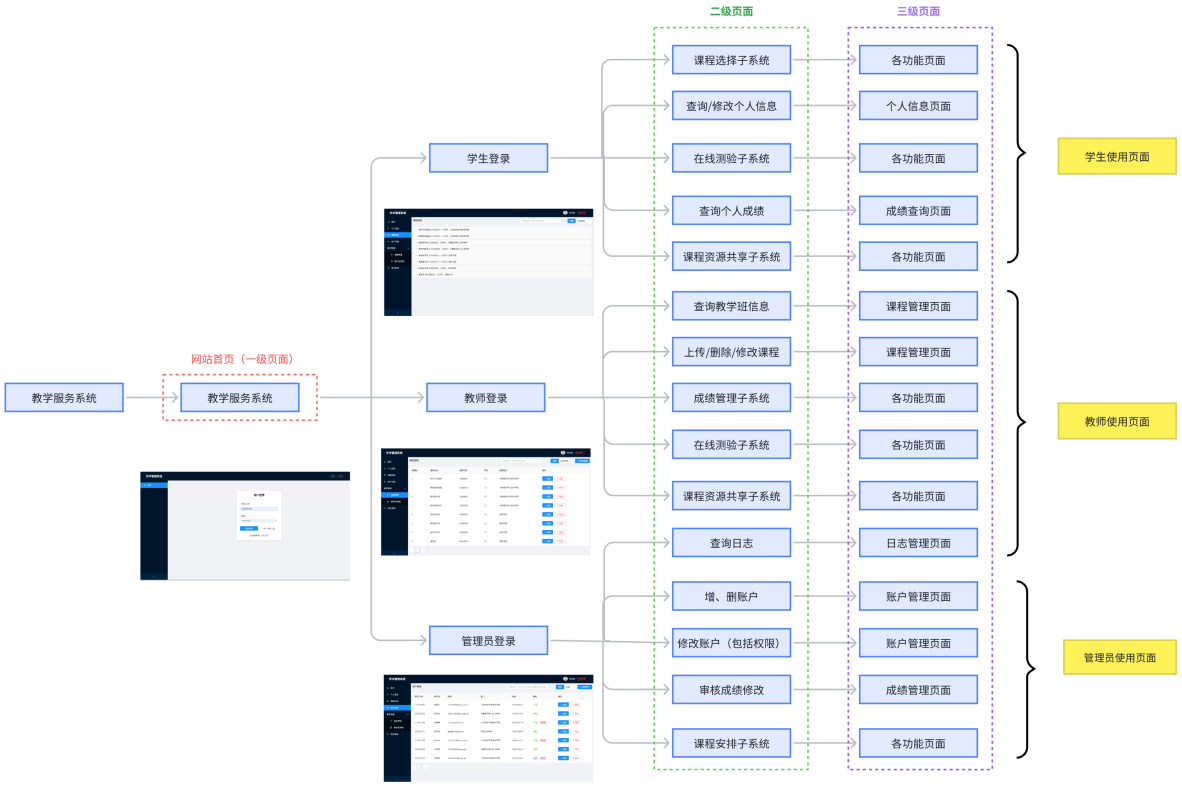


图 19: 系统整体结构图

功能模块图：

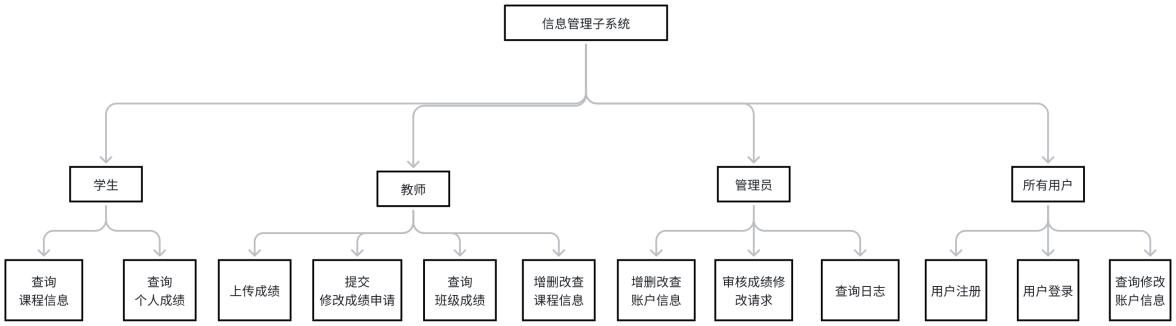


图 20: 功能模块图

系统分层图：

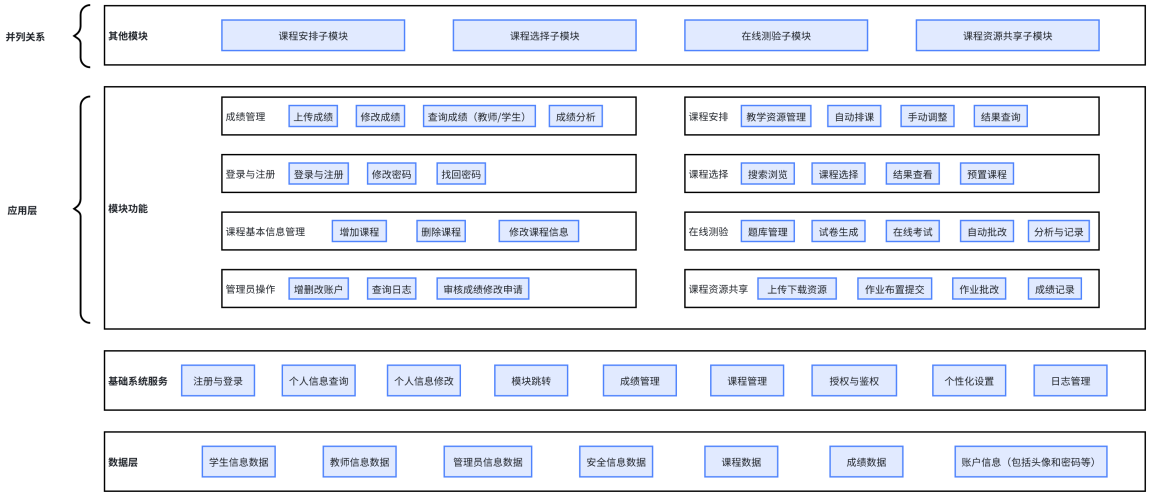


图 21: 系统分层图

3.4 技术介绍

3.4.1 前端框架

在系统设计中，我们将采用基于 HTML、CSS 和 C# 的网络前端技术，并使用 Blazor 框架实现服务客户端页面的构建。Blazor 是一个现代的.NET 前端 Web 框架，支持基于组件的开发模式，允许开发者使用 C# 和 Razor 模板语言构建交互式 Web 用户界面，而无需编写 JavaScript。Blazor 同时支持服务器端渲染（Blazor Server）和客户端 WebAssembly 部署（Blazor WebAssembly），适配多种运行环境和性能需求，根据项目需求我们最终选择了客户端 WebAssembly 部署。此外，我们还引入了 Ant Design Blazor 组件库，用于实现美观一致的前端界面，同时使用 Blazor ECharts 图表库完成成绩数据的可视化展示，从而提升用户的交互体验和数据感知能力。

3.4.2 后端框架

同时，我们采用 ASP.NET Core 作为后端开发框架。ASP.NET Core 是由微软开发的、用于构建高性能 Web 应用、API 和微服务的跨平台开源框架，可运行于 Windows、macOS 与 Linux 等多种操作系统。其两大核心特性——依赖注入（Dependency Injection）和面向切面编程（AOP）——大幅提升了项目的模块化程度和可维护性。ASP.NET Core 提供了灵活的中间件机制、强大的路由系统、内建安全支持等丰富功能，能够满足复杂业务系统的开发需求。

3.4.3 数据库

数据库方面，我们选用 PostgreSQL，它是一种特性非常齐全的自由软件的对象-关系型数据库管理系统（ORDBMS），是以加州大学计算机系开发的 POSTGRES，4.2 版本为基础的对象关系型数据库管理系统。PostgreSQL 支持复杂查询、事务管理、JSON 存储、并发控制等高级特性，能够为本系统提供高可靠性、高一致性的数据存储服务。

3.4.4 ORM 框架

为实现数据访问的对象映射，我们使用 Entity Framework Core (EF Core) 作为 ORM 框架。Entity Framework (EF) Core 是轻量化、可扩展、开源和跨平台版的常用 Entity Framework 数据访问技术，它提供了强类型 LINQ 查询能力，支持 Code First 与 Database First 开发方式，极大提高了开发效率，减少了手写 SQL 的出错率，以及有效避免了 SQL 注入的风险。

3.4.5 单元测试框架

在测试方面，系统采用 xUnit 作为单元测试框架。xUnit 是.NET 社区推荐的主流测试框架之一，支持灵活的测试生命周期管理与断言机制，能够有效保证系统逻辑的正确性与鲁棒性。

3.4.6 认证与鉴权

认证与鉴权部分，我们引入了 JWT（JSON Web Token）技术，通过无状态令牌实现客户端与服务器之间的安全通信。JWT 支持多平台认证机制，适合现代分布式系统的访问控制需求，能够简洁而安全地保障用户身份验证与权限分发过程。

3.5 部署图

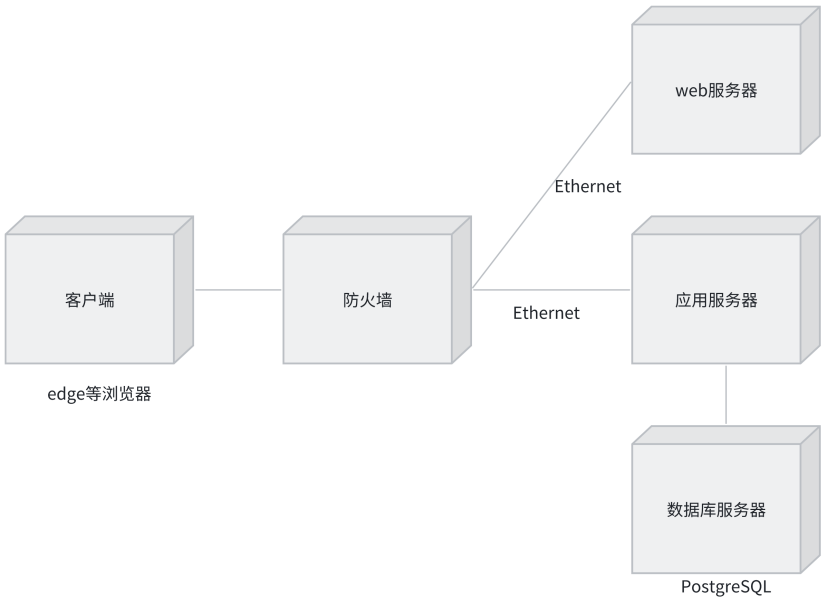


图 22: 部署图

3.6 架构语境图

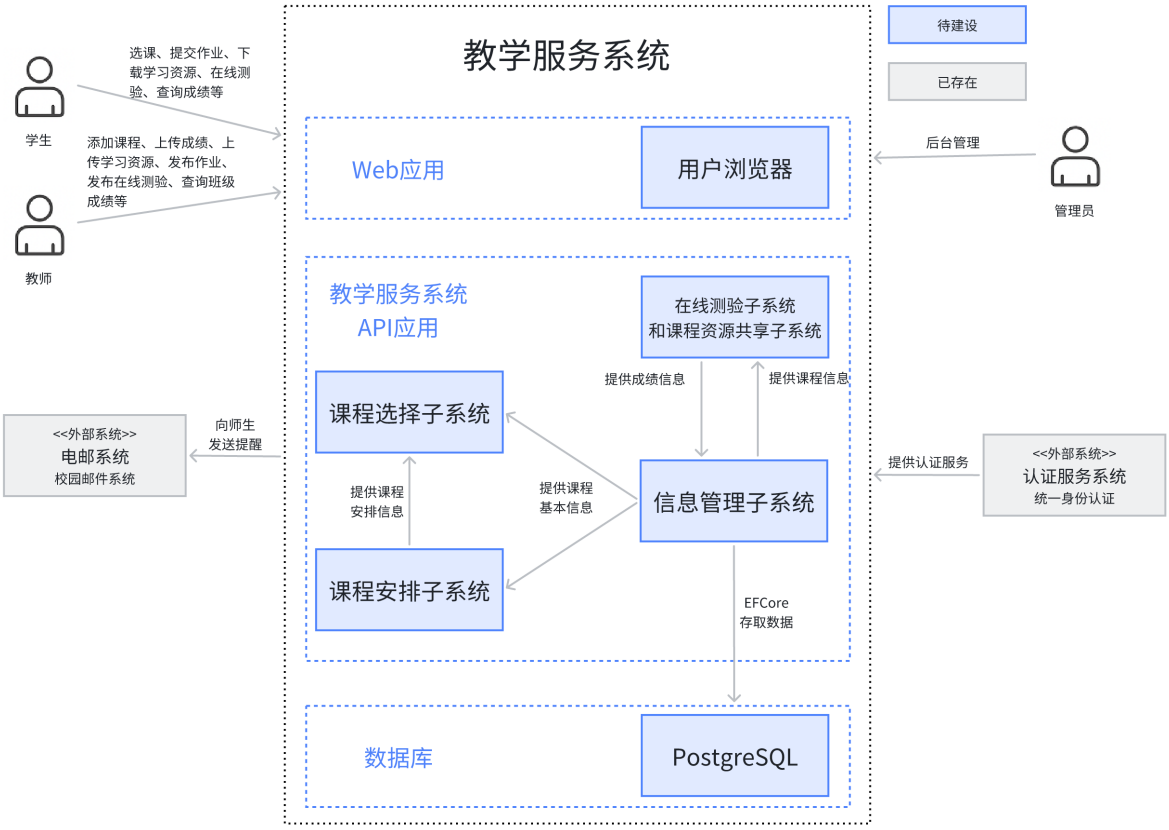


图 23: 架构语境图

3.7 类图

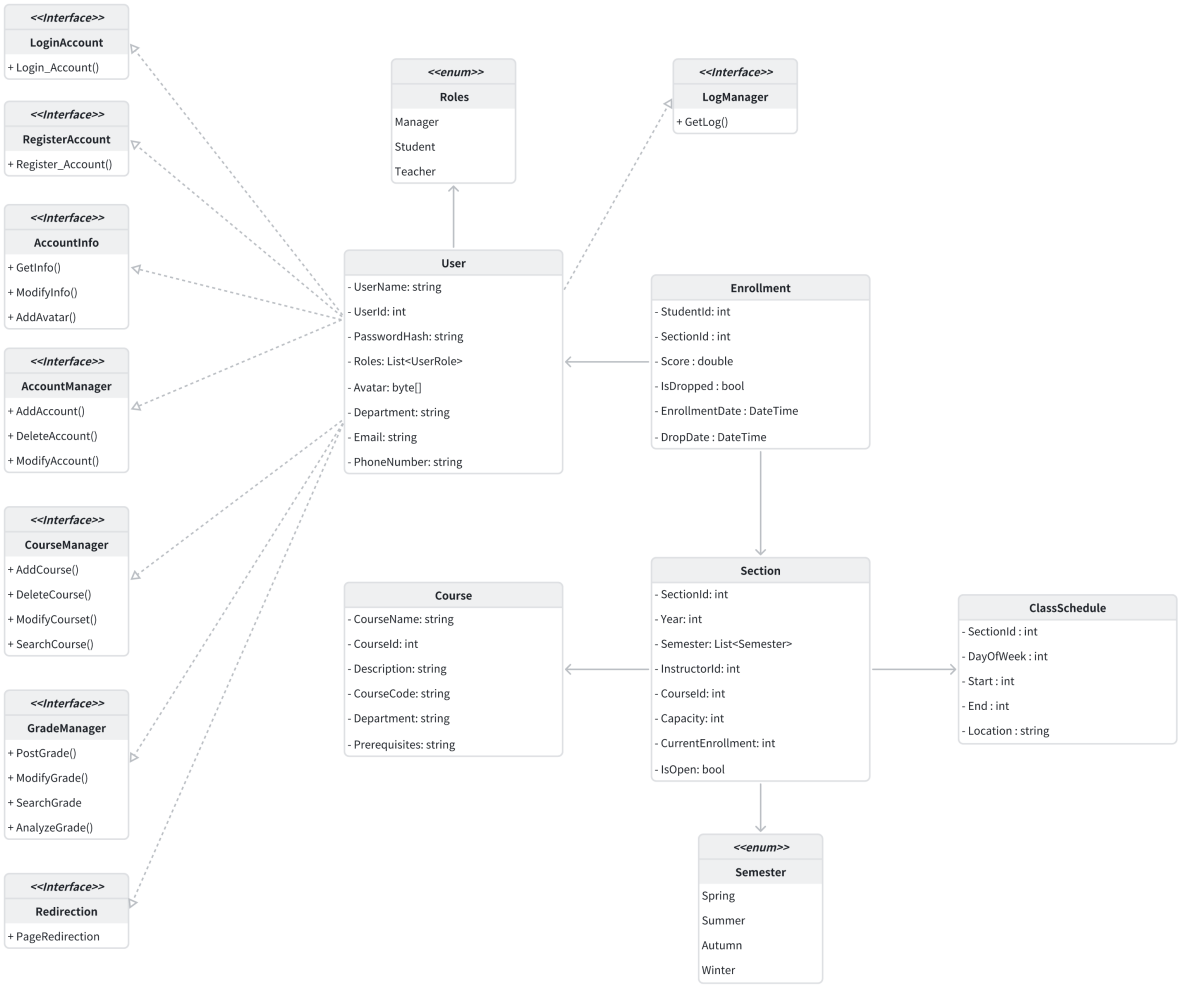


图 24: 类图

3.8 接口设计

3.8.1 内部接口

1. **LoginAccount**

   - `Login_Account(account: str, password: str) -> bool`
     用账户和密码登录，密码验证是对比哈希值，返回是否登录成功。

2. **RegisterAccount**

   - `Register_Account(account: str, password: str, identity_info: str) -> bool`
     注册新账户，包含账户、密码和身份信息，密码以哈希形式存储，返回是否注册成功。

3. **AccountInfo**

   - `GetInfo(user_id: int) -> Information`
     获取指定用户的账户信息。

- `ModifyInfo(user_id: int, new_info: Information) -> bool`
  修改用户的基本信息。
- `AddAvatar(user_id: int, image: bytes) -> bool`
  为用户添加或更换头像。

4. **AccountManager**

- `AddAccount(account: Account) -> bool`
  添加一个新账户。
- `DeleteAccount(user_id: int) -> bool`
  删除指定用户的账户。
- `ModifyAccount(user_id: int, account: Account) -> bool`
  修改指定用户的账户信息。

5. **CourseManager**

- `AddCourse(course: Course) -> bool`
  添加新课程。
- `DeleteCourse(course_id: int) -> bool`
  删除指定课程。
- `ModifyCourse(course_id: int, course: Course) -> bool`
  修改课程信息。
- `SearchCourse(keyword: str) -> List[Course]`
  通过关键词搜索课程。

6. **GradeManager**

- `PostGrade(grade: Grade) -> bool`
  发布成绩。
- `ModifyGrade(grade_id: int, grade: Grade) -> bool`
  修改成绩。
- `SearchGrade(student_id: int) -> List[Grade]`
  查询学生成绩。
- `AnalyzeGrade(course_id: int) -> AnalysisResult`
  对指定课程成绩进行分析。

7. **Redirection**

- `PageRedirection(target_url: str) -> None`
  实现页面跳转功能。

8. **LogManager**

- `GetLog() -> List[LogEntry]`
  获取系统日志记录，返回日志条目列表。

3.8.2 外部接口

a. 硬件接口：无 b. 软件接口：

1. **Endpoint:** `/login`
   - **Method:** POST
   - **Request Body:**
     - `account` (string)
     - `password` (string)
   - **Response:** Login state (bool)
2. **Endpoint:** `/register`
   - **Method:** POST
   - **Request Body:**
     - `account` (string)
     - `password` (string)
     - `identity_info` (string)
   - **Response:** Register state (bool)
3. **Endpoint:** `/account/get`
   - **Method:** GET
   - **Query Param:**
     - `user_id` (int)
   - **Response:** Account information (JSON)
4. **Endpoint:** `/account/modify`
   - **Method:** PUT
   - **Request Body:**
     - `user_id` (int)
     - `new_info` (JSON)
   - **Response:** Modify result (bool)
5. **Endpoint:** `/account/avatar`
   - **Method:** POST
   - **Request Body:**
     - `user_id` (int)
     - `image` (binary)
   - **Response:** Upload result (bool)
6. **Endpoint:** `/admin/account/add`

- **Method:** POST
- **Request Body:** Account (JSON)
- **Response:** Add result (bool)

7. **Endpoint:** `/admin/account/delete`
   - **Method:** DELETE
   - **Query Param:**
     - `user_id` (int)
   - **Response:** Delete result (bool)
8. **Endpoint:** `/admin/account/modify`
   - **Method:** PUT
   - **Request Body:**
     - `user_id` (int)
     - `account` (JSON)
   - **Response:** Modify result (bool)
9. **Endpoint:** `/course/add`
   - **Method:** POST
   - **Request Body:** Course (JSON)
   - **Response:** Add result (bool)
10. **Endpoint:** `/course/delete`
    - **Method:** DELETE
    - **Query Param:**
      - `course_id` (int)
    - **Response:** Delete result (bool)
11. **Endpoint:** `/course/modify`
    - **Method:** PUT
    - **Request Body:**
      - `course_id` (int)
      - `course` (JSON)
    - **Response:** Modify result (bool)
12. **Endpoint:** `/course/search`
    - **Method:** GET
    - **Query Param:**
      - `keyword` (string)

- **Response:** List of courses (JSON)

13. **Endpoint:** `/grade/post`
    - **Method:** POST
    - **Request Body:** Grade (JSON)
    - **Response:** Post result (bool)
14. **Endpoint:** `/grade/modify`
    - **Method:** PUT
    - **Request Body:**
      - `grade_id` (int)
      - `grade` (JSON)
    - **Response:** Modify result (bool)
15. **Endpoint:** `/grade/search`
    - **Method:** GET
    - **Query Param:**
      - `student_id` (int)
    - **Response:** List of grades (JSON)
16. **Endpoint:** `/grade/analyze`
    - **Method:** GET
    - **Query Param:**
      - `course_id` (int)
    - **Response:** Analysis result (JSON)
17. **Endpoint:** `/redirect`
    - **Method:** GET
    - **Query Param:**
      - `target_url` (string)
    - **Response:** HTTP Redirect
18. **Endpoint:** `/log/get`
    - **Method:** GET
    - **Query Param:** 无（可选支持分页参数，如 page/size）
    - **Response:** List of log entries (JSON)

4 详细设计

4.1 顺序图

4.1.1 登陆账户

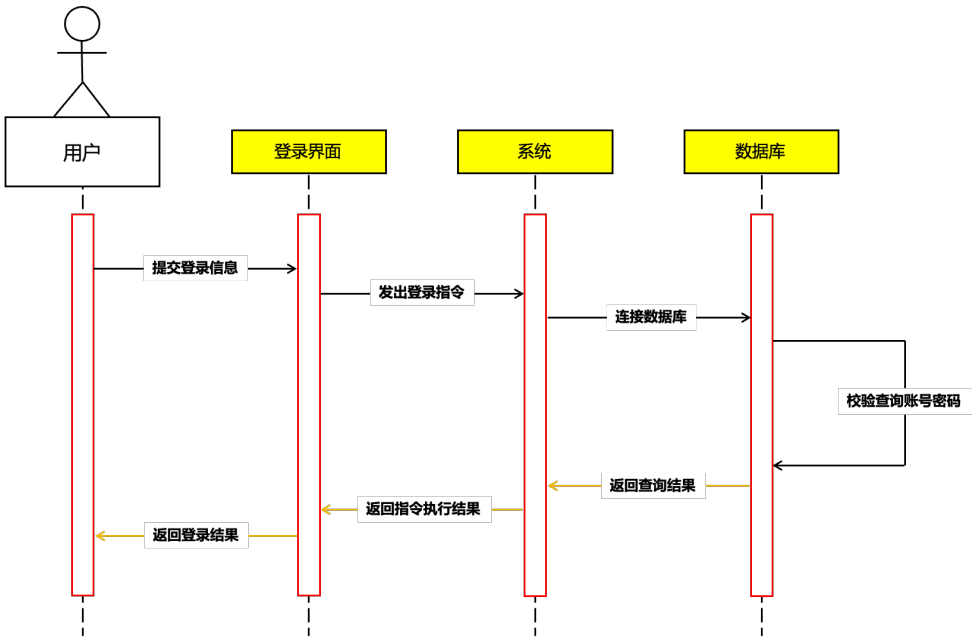


图 25: 登录账户顺序图

4.1.2 注册账户

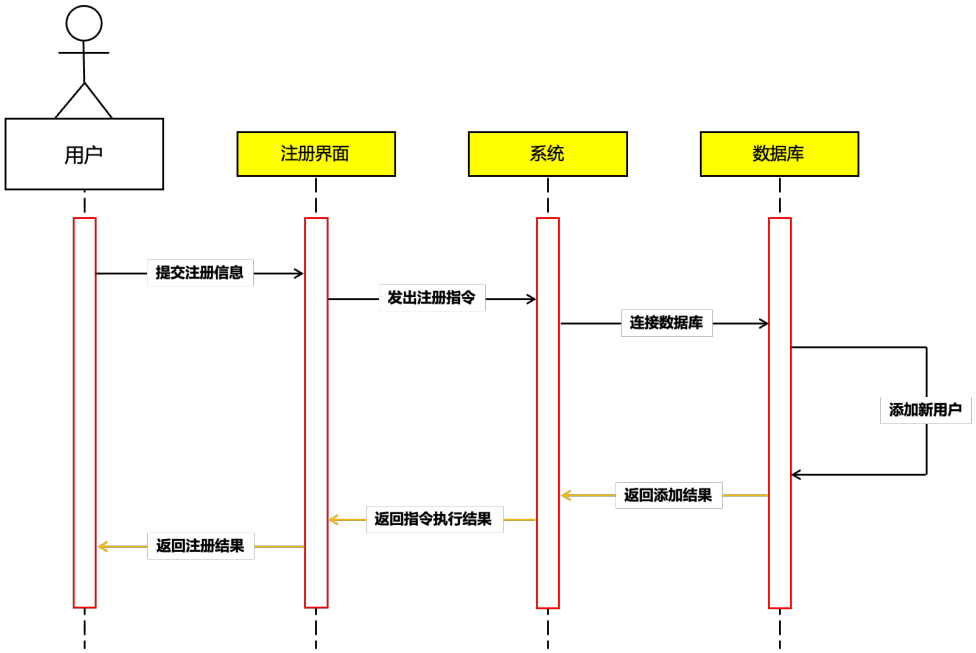


图 26: 注册账户顺序图

4.1.3 管理员增删改账户信息

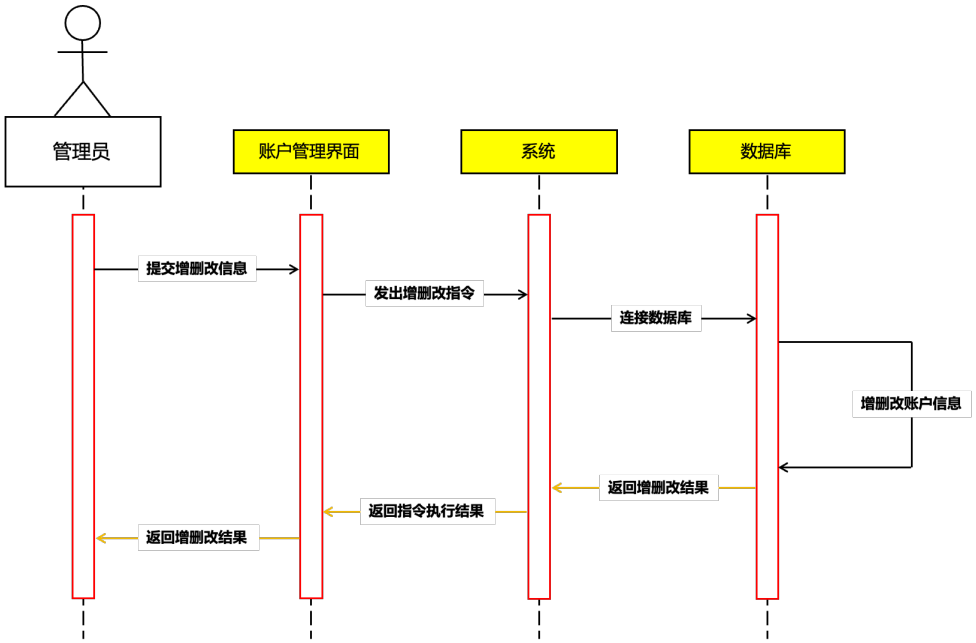


图 27: 管理员增删账户信息顺序图

4.1.4 查询个人信息

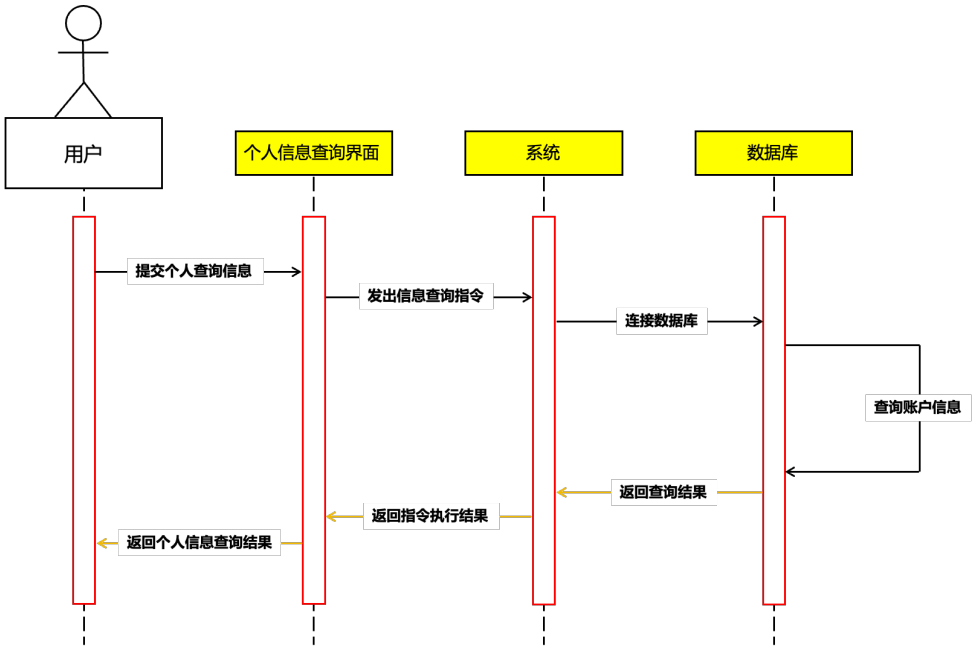


图 28: 查询个人信息顺序图

4.1.5 修改个人信息

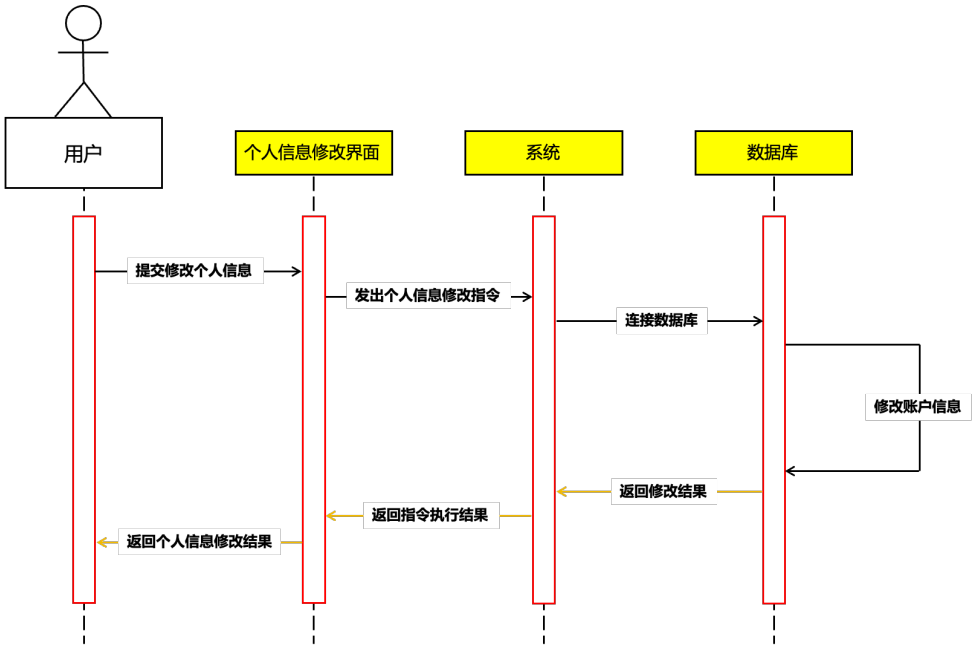


图 29: 修改个人信息顺序图

4.1.6 教师增删改课程信息

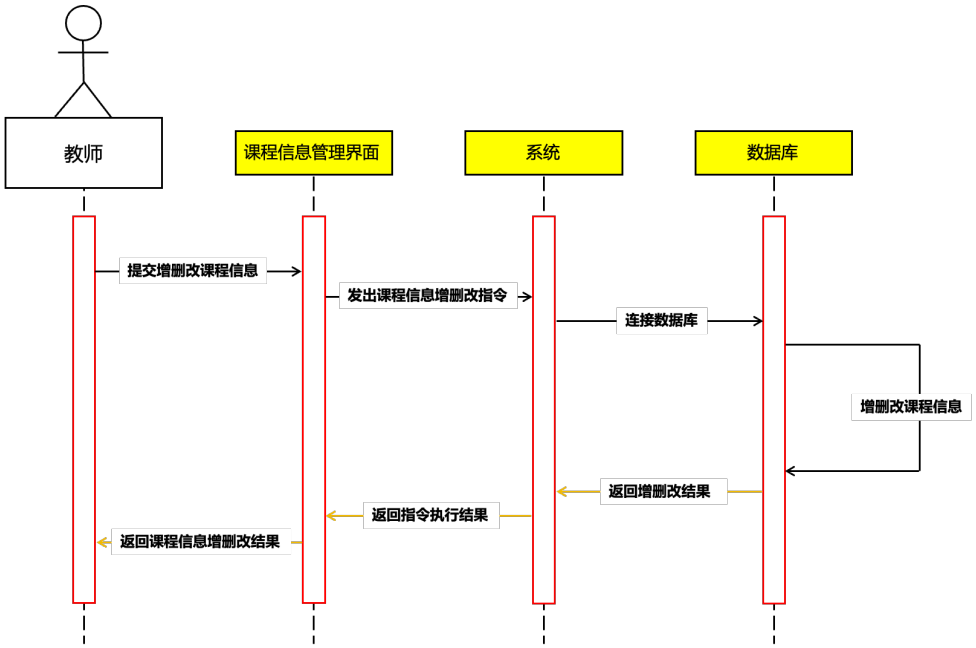


图 30: 教师增删改课程信息顺序图

4.1.7 查询课程信息

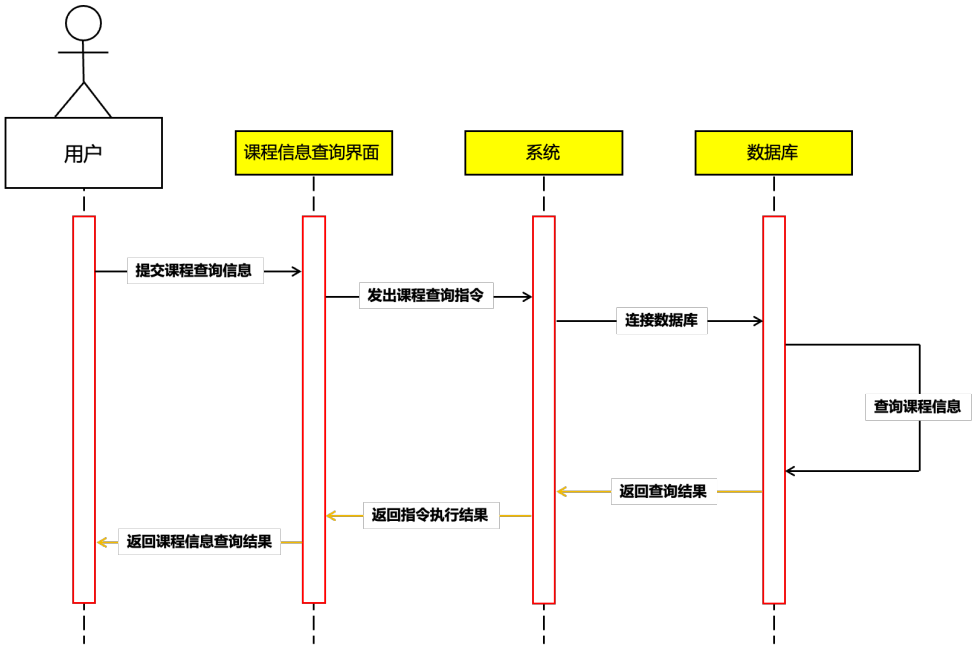


图 31: 查询课程顺序图

4.1.8 教师上传成绩

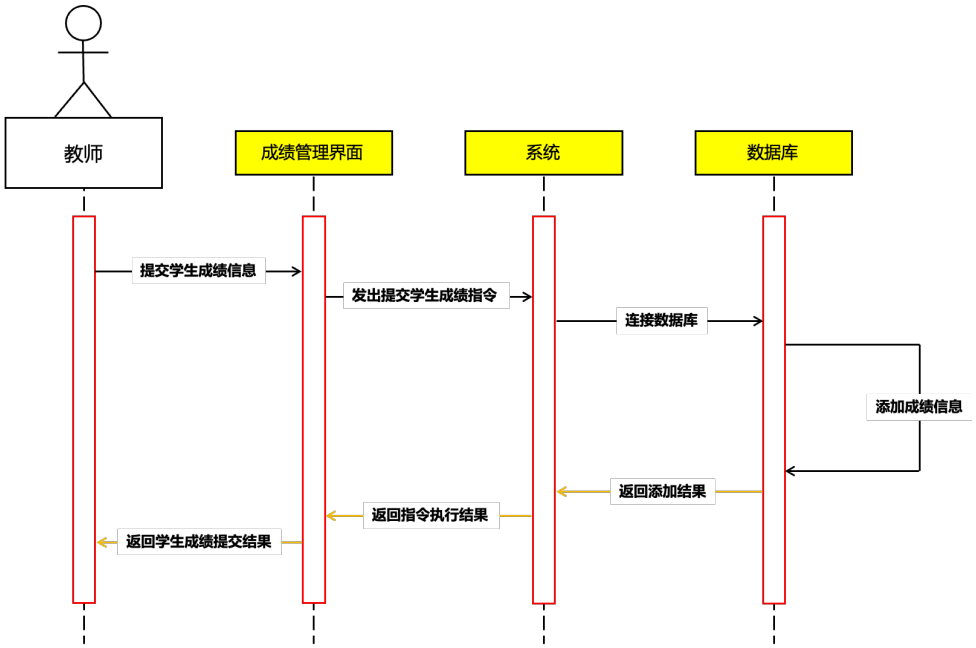


图 32: 教师上传成绩顺序图

4.1.9 教师/学生查询成绩

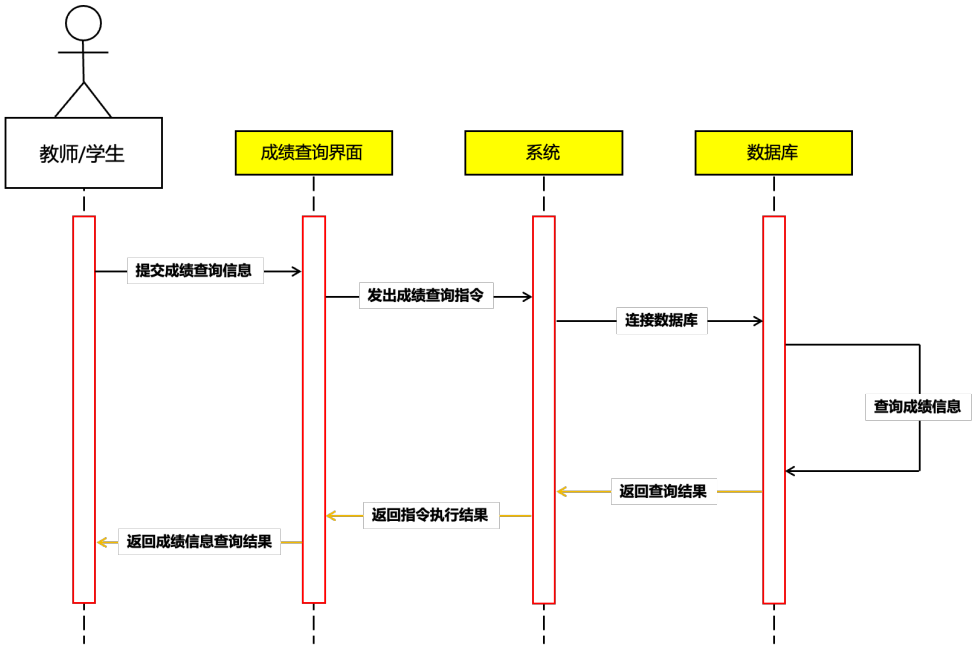


图 33: 教师/学生查询成绩顺序图

4.1.10 修改成绩

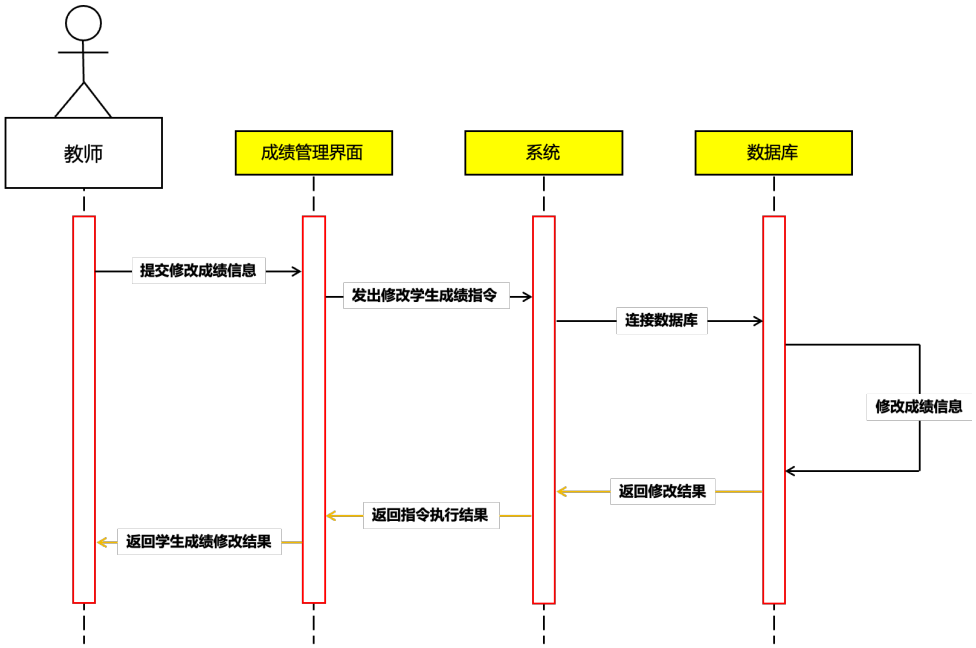


图 34: 修改成绩顺序图

4.2 执行概念

4.2.1 登录账户

4.2.1.1 模块概述

在该模块中，用户可输入登录信息，系统将其与数据库中的信息进行比对，以验证用户身份，确定是否允许用户登录教务管理系统的信息管理子系统。登录信息用于身份验证，若验证通过，用户可正常使用系统功能。

4.2.1.2 IPO 图

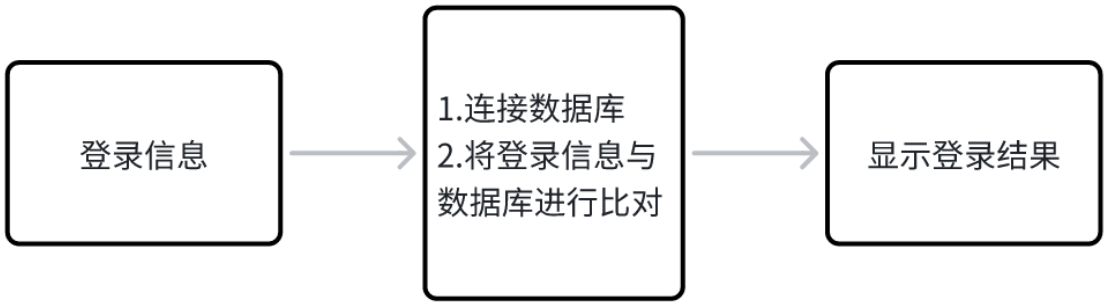


图 35: 登录 IPO 图

4.2.1.3 功能

实现用户登录账户功能，验证用户输入的登录信息，决定是否允许用户进入教务管理系统的信息管理子系统。

4.2.1.4 输入项

名称	标识	类型和格式	输入方式
用户名	username	string	外部输入（用户在登录界面输入）
学号	student_id	string	外部输入（用户在登录界面输入）
密码	password	string	外部输入（用户在登录界面输入）

表 7: 登录信息输入项

4.2.1.5 输出项

名称	标识	类型和格式	输出方式
登录结果	login_result	boolean	由脚本输出
跳转页面	redirect_page	string	由脚本输出（登录成功为系统主界面地址，失败为登录页面地址）

表 8: 登录响应输出项

4.2.1.6 设计方法（算法）

```
while（用户未完成登录信息输入）:
    获取用户输入的用户名和密码
if（用户名和密码与数据库中记录匹配）:
    显示登录成功消息
    将用户跳转至系统主界面
    返回登录成功
else:
    显示用户名或密码错误消息
    留在登录页面
    返回登录失败
```

4.2.1.7 流程图

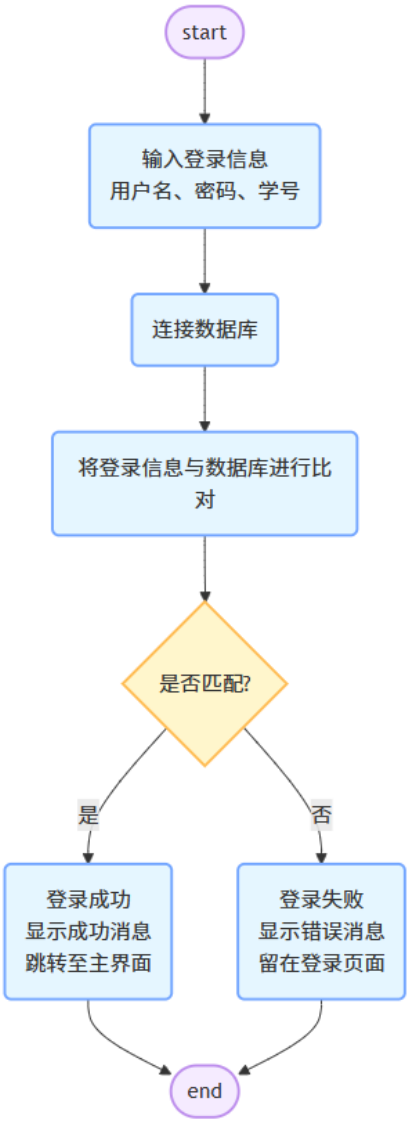


图 36: 用户登录流程图

4.2.2 注册账户

4.2.2.1 模块概述

在本模块中，用户可通过输入姓名、学号等进行注册，或通过统一身份认证并绑定个人手机号。所输入信息有特定长度要求，注册信息将用于后续密码修改与解冻时的身份验证。通过该模块实现新用户在教务管理系统信息管理子系统中的注册功能。

4.2.2.2 IPO 图

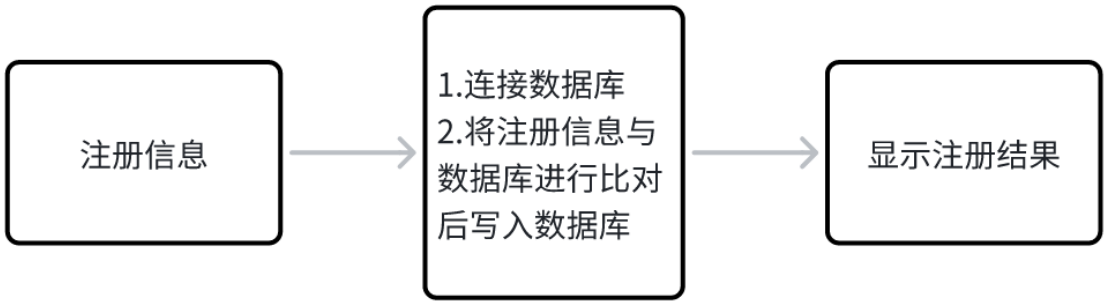


图 37: 注册 IPO 图

4.2.2.3 功能

实现用户注册并绑定账号功能，将用户注册信息存入数据库，同时反馈注册操作结果给用户。

4.2.2.4 输入项

名称	标识	类型和格式	输入方式
姓名	name	string	外部输入（用户在注册界面输入）
电话号码	phone_number	string	外部输入（用户在注册界面输入）
学号	student_id	string	外部输入（用户在注册界面输入）

表 9: 注册信息输入项

4.2.2.5 输出项

名称	标识	类型和格式	输出方式
注册结果	register_result	boolean	由脚本输出
账号	account_number	string	由脚本输出（注册成功时提供）
失败提示	failure_tip	string	由脚本输出（注册失败时提供）

表 10: 注册响应输出项

4.2.2.6 设计方法（算法）

```
while (用户未完成注册信息输入):
    获取用户输入的姓名、电话号码、学号等注册信息
# 连接数据库
connect_database()
# 检查学号、电话号码是否已存在于数据库
if (学号已存在于数据库 or 电话号码已存在于数据库):
    显示注册失败消息，提示信息已被占用
    返回注册失败
else:
    将用户注册信息写入数据库
    生成账号
    显示注册成功消息，提供账号
    返回注册成功
```

4.2.2.7 流程图

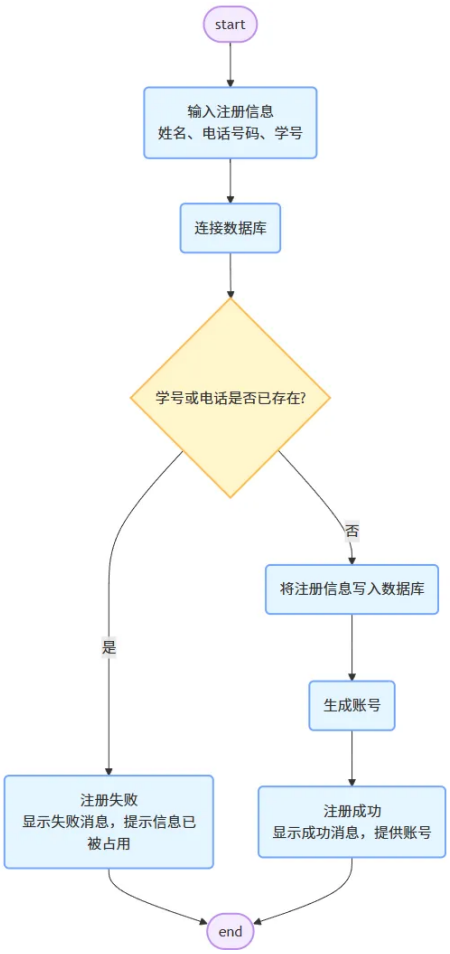


图 38: 用户注册流程图

4.2.3 管理员增删改账户信息

4.2.3.1 模块概述

此模块赋予管理员对账户信息进行增加、删除、修改操作的权限。管理员通过特定操作界面输入相关指令和信息，系统连接数据库并依据指令执行相应操作，最终将操作结果反馈给管理员，确

保账户信息能按需更新维护。

4.2.3.2 IPO 图

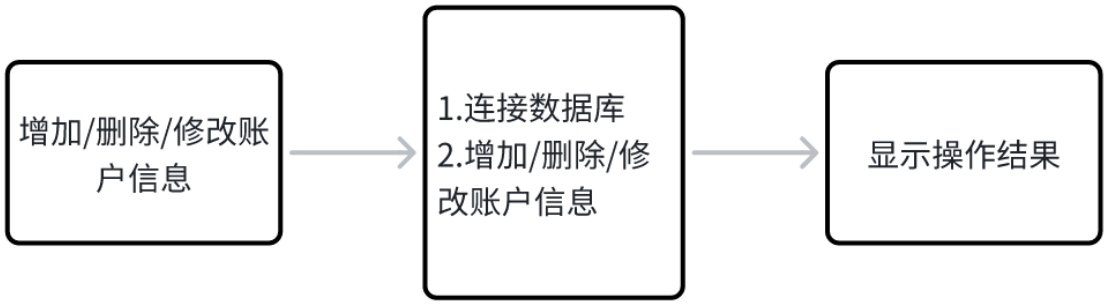


图 39: 管理员增删改账户信息 IPO 图

4.2.3.3 功能

实现管理员对教务管理系统信息管理子系统中账户信息的增删改操作，保证系统账户数据准确、完整且符合管理需求。

4.2.3.4 输入项

名称	标识	类型和格式	输入方式
操作指令	operation_command	string （取值为“增加”“删除”“修改”）	外部输入（管理员在操作界面选择输入）
新增账户信息（若为增加操作）	new_account_info	结构化数据（包含用户名、密码、身份证号等相关字段，格式依数据库表结构）	外部输入（管理员在操作界面输入）
需删除账户标识（若为删除操作）	delete_account_identif	string （如用户名、账号等唯一标识）	外部输入（管理员在操作界面输入）
需修改账户信息及修改内容（若为修改操作）	modify_account_info modification_content	结构化数据（前者为账户原信息标识，后者为修改后内容，格式依数据库表结构）	外部输入（管理员在操作界面输入）

表 11: 管理员增删改账户信息输入项

4.2.3.5 输出项

名称	标识	类型和格式	输出方式
操作结果	operation_result	boolean	由脚本输出
失败提示	failure_tip	string	由脚本输出（操作失败时提供）

表 12: 管理员增删改账户信息输出项

4.2.3.6 设计方法（算法）

```
# 获取管理员输入的操作指令及相关信息
operation_command = 获取操作指令()
if operation_command == "增加":
    new_account_info = 获取新增账户信息()
    connect_database()
    if 数据库中不存在相同标识账户(new_account_info):
        将新账户信息写入数据库(new_account_info)
        显示操作成功消息
        返回操作成功
    else:
        显示操作失败消息，提示账户已存在
        返回操作失败
elif operation_command == "删除":
    delete_account_identifier = 获取需删除账户标识()
    connect_database()
    if 数据库中存在该账户(delete_account_identifier):
        从数据库删除账户(delete_account_identifier)
        显示操作成功消息
        返回操作成功
    else:
        显示操作失败消息，提示账户不存在
        返回操作失败
elif operation_command == "修改":
    modify_account_info, modification_content = 获取需修改账户信息及修改内容()
    connect_database()
    if 数据库中存在该账户(modify_account_info):
        在数据库更新账户信息(modify_account_info, modification_content)
        显示操作成功消息
        返回操作成功
    else:
        显示操作失败消息，提示账户不存在
        返回操作失败
```

4.2.3.7 流程图

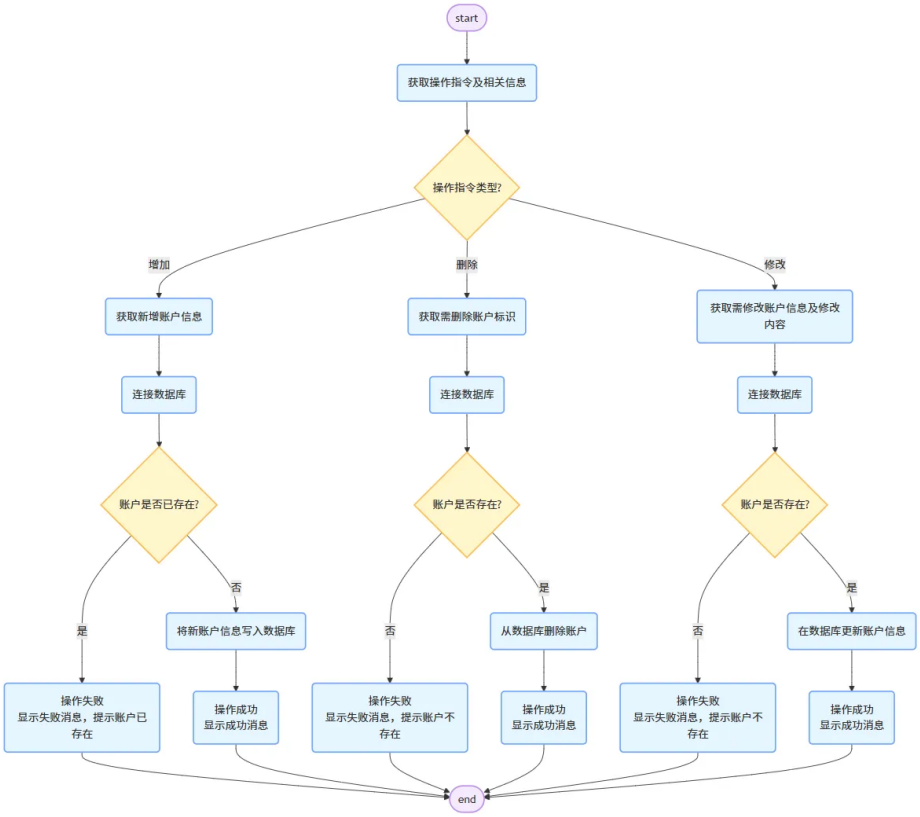


图 40: 管理员增删改账户信息流程图

4.2.4 查询个人信息

4.2.4.1 模块概述

此模块允许用户（或管理员在特定权限下）查询账户相关的个人信息。用户通过操作界面发起查询请求，系统连接数据库并依据用户身份标识（如用户名、账号等）检索相关信息，最后将查询结果展示给用户，方便用户了解自身账户详情。

4.2.4.2 IPO 图

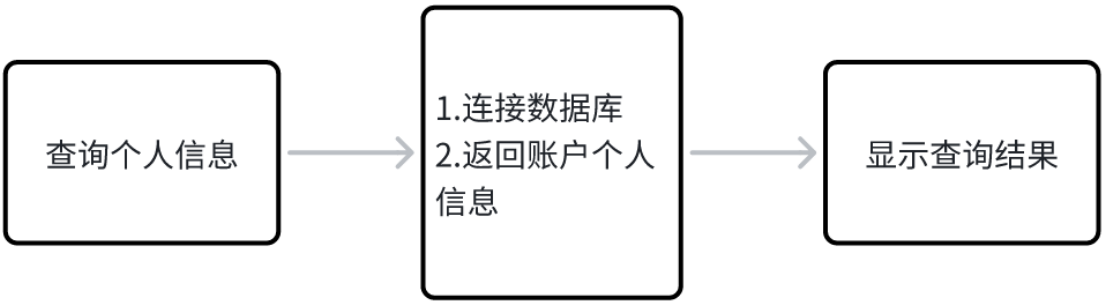


图 41: 查询个人信息 IPO 图

4.2.4.3 功能

实现对用户账户个人信息的查询功能，为用户提供便捷的信息获取途径，确保用户能及时准确

获取自身账户相关信息。

4.2.4.4 输入项

名称	标识	类型和格式	输入方式
用户身份标识	user_identifier	string （如用户名、账号）	外部输入（用户在查询界面输入或系统自动获取已登录用户标识）

表 13: 查询个人信息输入项

4.2.4.5 输出项

名称	标识	类型和格式	输出方式
查询结果	query_result	结构化数据（依数据库中个人信息表结构，包含姓名、联系方式等字段）	由脚本输出并展示在查询结果页面

表 14: 查询个人信息输出项

4.2.4.6 设计方法（算法）

```
# 这里的算法似乎与查询功能不匹配，重新编写符合查询功能的算法
user_identifier = 获取用户身份标识()
connect_database()
执行查询语句 "SELECT * FROM user_info WHERE identifier =?" WITH PARAMETERS (
    user_identifier)
query_result = FETCH QUERY RESULT
if (query_result 存在):
    展示查询结果
else:
    显示未查询到相关信息提示
close_database_connection()
```

4.2.4.7 流程图

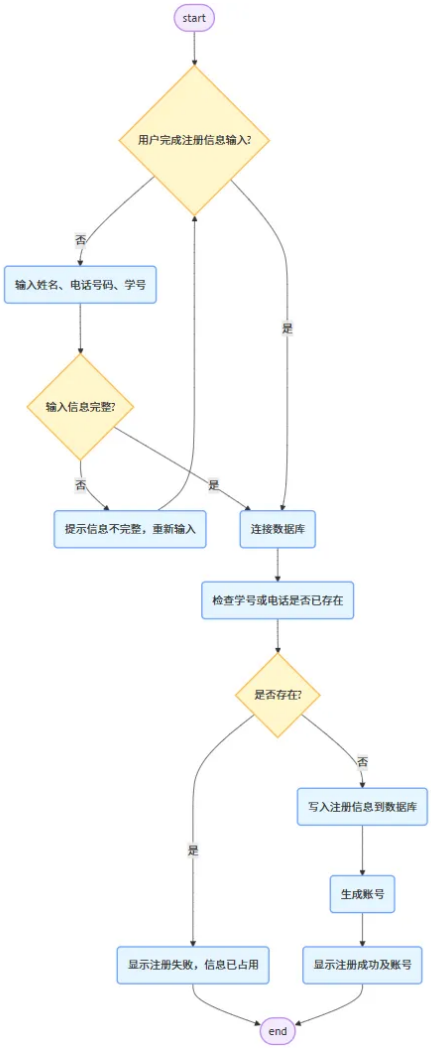


图 42: 查询个人信息流程图

4.2.5 修改个人信息

4.2.5.1 模块概述

该模块允许用户对自身账户的个人信息进行修改，如姓名、手机号码、学号等。用户通过系统界面提交修改请求及新的信息内容，系统连接数据库，依据用户身份标识定位到对应账户记录，并将新信息更新至数据库中，最后向用户反馈修改操作的结果。

4.2.5.2 IPO 图

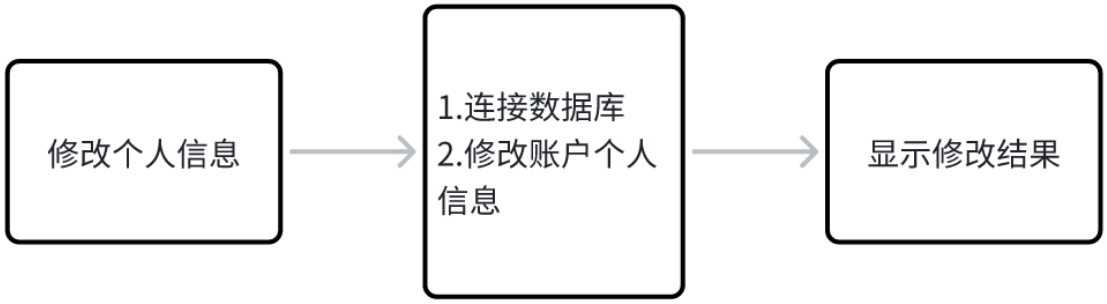


图 43: 修改个人信息 IPO 图

4.2.5.3 功能

实现用户对个人信息的自主修改功能，确保用户账户信息的准确性和及时性，满足用户信息变更的需求。

4.2.5.4 输入项

名称	标识	类型和格式	输入方式
用户身份标识	user_identifier	string（如姓名、学号）	外部输入（系统自动获取已登录用户标识或用户手动输入）
需修改的信息项及新内容	modified_info_items new_content	结构化数据（如字典形式，包含键值对，键为信息项名称，值为新内容，如”name”: ” 新姓名”, ”phone”: ” 新电话号码” ）	外部输入（用户在修改界面输入）

表 15: 修改个人信息输入项

4.2.5.5 输出项

名称	标识	类型和格式	输出方式
修改结果	modification_result	boolean	由脚本输出
失败提示	failure_tip	string	由脚本输出（修改失败时提供）

表 16: 修改个人信息输出项

4.2.5.6 设计方法（算法）

```
# 循环，直到用户完成修改信息输入
while (用户未完成修改信息输入):
    # 获取用户输入的身份标识、需修改的信息项及新内容
    用户身份标识 = 输入用户身份标识()
    需修改的信息项及新内容 = 输入需修改的信息项及新内容()

    # 这里可添加对输入信息的简单验证，如判断是否为空
    if (用户身份标识 为空 OR 需修改的信息项及新内容 为空):
        显示输入信息不完整，请重新输入提示()
    else:
        break  # 跳出循环，认为用户完成输入

# 连接数据库
connect_database()

# 构建 SQL 更新语句
UPDATE_QUERY = "UPDATE users SET "
# 遍历需修改的信息项
for 信息项 in 需修改的信息项及新内容:
    UPDATE_QUERY = UPDATE_QUERY + 信息项 + " = '" + 需修改的信息项及新内容[信息项]
        + "', "
# 去除最后多余的逗号和空格
UPDATE_QUERY = UPDATE_QUERY.rstrip(", ")
UPDATE_QUERY = UPDATE_QUERY + " WHERE username = '" + 用户身份标识 + "'"

# 执行 SQL 更新语句
执行(UPDATE_QUERY)

# 提交数据库事务
commit_database_transaction()

# 检查更新是否成功（可根据数据库返回结果判断）
if (更新成功):
    显示个人信息修改成功消息()
    return 修改成功
else:
    显示个人信息修改失败消息，提示失败原因()
    return 修改失败

# 关闭数据库连接
close_database_connection()
```

4.2.5.7 流程图

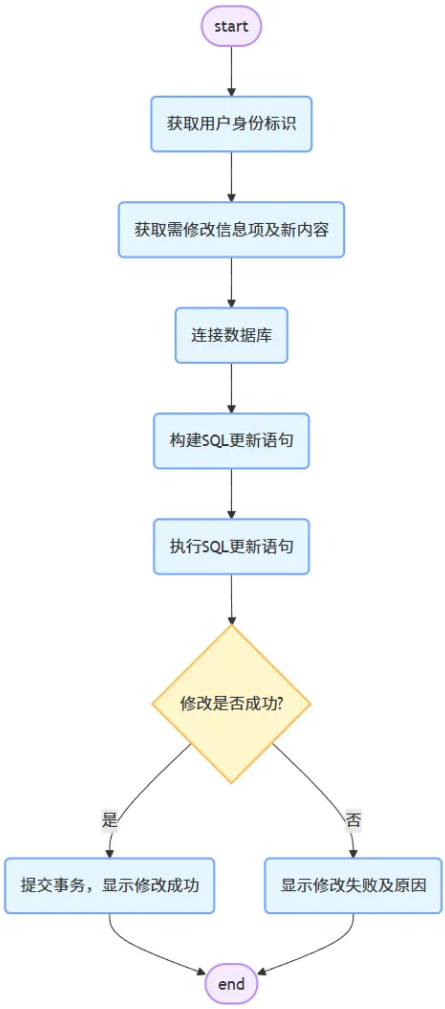


图 44: 修改个人信息流程图

4.2.6 教师增删改课程信息

4.2.6.1 模块概述

该模块为教师提供对课程信息进行增加、删除、修改操作的功能。教师通过系统操作界面选择相应操作类型，并输入或选择与课程相关的具体信息（如课程名称、学分、课时等）。系统连接数据库，根据教师的操作指令对课程信息表进行相应的数据操作，最后将操作结果反馈给教师，以保证课程信息的准确性和实时更新。

4.2.6.2 IPO 图

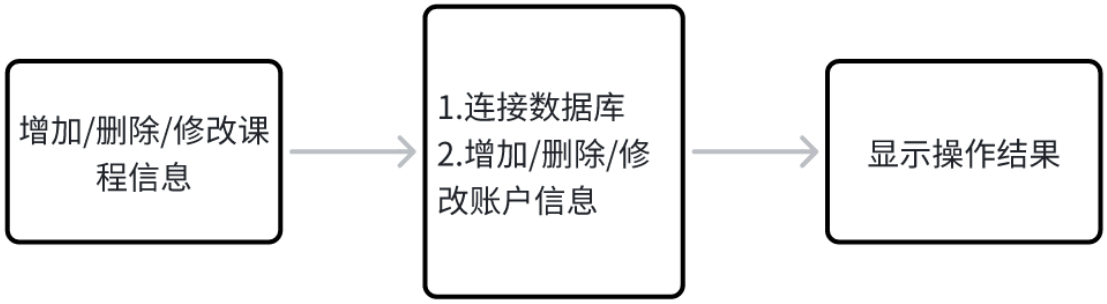


图 45: 教师增删改课程信息 IPO 图

4.2.6.3 功能

实现教师对所授课程信息的管理功能，确保课程信息在教务管理系统中准确无误，满足教学安排和管理的需求。

4.2.6.4 输入项

名称	标识	类型和格式	输入方式
操作指令	operation_command	string （取值为“增加”“删除”“修改”）	外部输入（教师在操作界面选择）
课程编号（若为删除或修改操作）	course_id	string（唯一标识课程）	外部输入（教师在操作界面输入或选择）
课程名称	course_name	string	外部输入（教师在操作界面输入，增加或修改操作时需提供）
学分	credit	float	外部输入（教师在操作界面输入，增加或修改操作时需提供）
课时	class_hours	int	外部输入（教师在操作界面输入，增加或修改操作时需提供）

表 17: 教师增删改课程信息输入项

4.2.6.5 输出项

名称	标识	类型和格式	输出方式
操作结果	operation_result	boolean	由脚本输出
失败提示	failure_tip	string	由脚本输出（操作失败时提供）

表 18: 教师增删改课程信息输出项

4.2.6.6 设计方法（算法）

```
# 循环，直到教师完成操作信息输入
while (教师未完成操作信息输入):
    # 获取教师输入的操作指令
    操作指令 = 输入操作指令()
    if (操作指令 为空):
        显示请输入有效的操作指令提示()
    elif (操作指令 not in ["增加", "删除", "修改"]):
        显示操作指令无效，请输入增加、删除或修改提示()
    else:
        if (操作指令 == "增加"):
            # 获取新增课程信息
            课程名称 = 输入课程名称()
            学分 = 输入学分()
            课时 = 输入课时()
            if (课程名称 为空 or 学分 为空 or 课时 为空):
                显示输入信息不完整，请重新输入提示()
            else:
                break  # 跳出循环，认为教师完成输入
        elif (操作指令 == "删除"):
            # 获取需删除课程的编号
            课程编号 = 输入课程编号()
            if (课程编号 为空):
                显示请输入有效的课程编号提示()
            else:
                break  # 跳出循环，认为教师完成输入
        elif (操作指令 == "修改"):
            # 获取需修改课程的编号
            课程编号 = 输入课程编号()
            if (课程编号 为空):
                显示请输入有效的课程编号提示()
            else:
                # 获取修改后的课程信息
                新的课程名称 = 输入新的课程名称()
                新的学分 = 输入新的学分()
                新的课时 = 输入新的课时()
                if (新的课程名称 为空 and 新的学分 为空 and 新的课时 为空):
                    显示至少输入一项要修改的信息提示()
                else:
                    break  # 跳出循环，认为教师完成输入
```

```
# 连接数据库
connect_database()

if (操作指令 == "增加"):
    # 检查课程名称是否已存在（假设课程名称唯一）
    执行查询语句 "SELECT * FROM courses WHERE course_name =?" WITH PARAMETERS (课程名称)
    result = FETCH_QUERY_RESULT
    if (result 存在):
        显示操作失败，课程名称已存在提示()
        return 操作失败
    else:
        # 将新增课程信息写入数据库
        执行插入语句 "INSERT INTO courses (course_name, credit, class_hours) VALUES (?,?,?)" WITH PARAMETERS (课程名称, 学分, 课时)
        commit_database_transaction()
        显示课程信息增加成功提示()
        return 操作成功
elif (操作指令 == "删除"):
    # 检查课程是否存在
    执行查询语句 "SELECT * FROM courses WHERE course_id =?" WITH PARAMETERS (课程编号)
    result = FETCH_QUERY_RESULT
    if (result 存在):
        # 从数据库删除课程
        执行删除语句 "DELETE FROM courses WHERE course_id =?" WITH PARAMETERS (课程编号)
        commit_database_transaction()
        显示课程信息删除成功提示()
        return 操作成功
    else:
        显示操作失败，课程不存在提示()
        return 操作失败
elif (操作指令 == "修改"):
    # 检查课程是否存在
    执行查询语句 "SELECT * FROM courses WHERE course_id =?" WITH PARAMETERS (课程编号)
    result = FETCH_QUERY_RESULT
    if (result 存在):
        # 构建 SQL 更新语句
        UPDATE_QUERY = "UPDATE courses SET "
        if (新的课程名称 不为空):
            UPDATE_QUERY = UPDATE_QUERY + "course_name = '" + 新的课程名称 + "', "
        if (新的学分 不为空):
            UPDATE_QUERY = UPDATE_QUERY + "credit = " + 新的学分 + ", "
        if (新的课时 不为空):
            UPDATE_QUERY = UPDATE_QUERY + "class_hours = " + 新的课时 + ", "
        # 去除最后多余的逗号和空格
        UPDATE_QUERY = UPDATE_QUERY.rstrip(", ")
        UPDATE_QUERY = UPDATE_QUERY + " WHERE course_id = '" + 课程编号 + "'"
```

```
        # 执行 SQL 更新语句
        执行(UPDATE_QUERY)
        commit_database_transaction()
        显示课程信息修改成功提示()
        return 操作成功
    else:
        显示操作失败，课程不存在提示()
        return 操作失败

# 关闭数据库连接
close_database_connection()
```

4.2.6.7 流程图

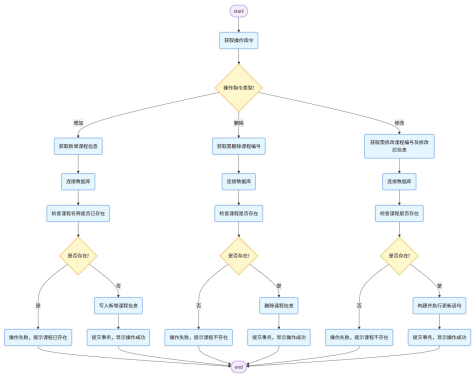


图 46: 教师增删改课程信息流程图

4.2.7 查询课程信息

4.2.7.1 模块概述

该模块为用户（包括教师、学生、管理员等）提供查询课程信息的功能。用户通过系统界面输入查询条件（如课程名称、课程编号等），系统连接数据库，依据输入条件在课程信息表中查找对应的课程信息，并将查询结果展示给用户，方便用户获取课程相关详情，辅助教学安排、选课等活动。

4.2.7.2 IPO 图

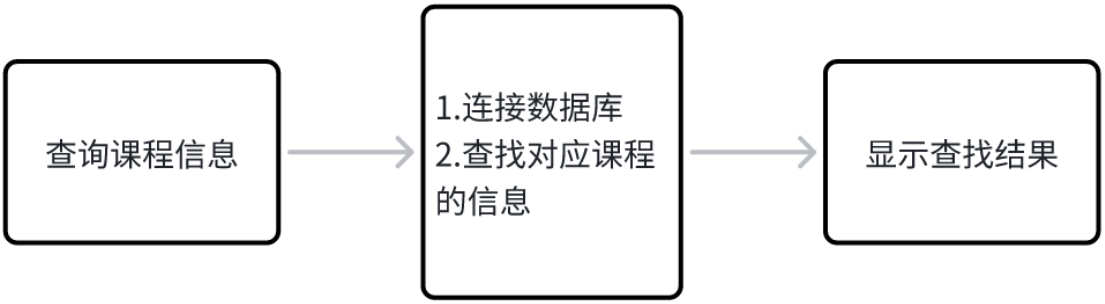


图 47: 查询课程信息 IPO 图

4.2.7.3 功能

实现对课程信息的查询功能，为不同用户角色提供便捷的课程信息获取途径，确保用户能及时、准确地获取课程相关数据。

4.2.7.4 输入项

名称	标识	类型和格式	输入方式
课程名称	course_name	string （可为空）	外部输入（用户在查询界面输入）
课程编号	course_id	string （可为空）	外部输入（用户在查询界面输入）

表 19: 查询课程信息输入项

4.2.7.5 输出项

名称	标识	类型和格式	输出方式
查询结果	query_result	结构化数据（依数据库课程信息表结构，包含课程名称、学分、课时等字段）	由脚本输出并展示在查询结果页面
提示信息	tip_info	string（未查询到结果时为“无匹配课程”等提示）	由脚本输出并展示在查询结果页面

表 20: 查询课程信息输出项

4.2.7.6 设计方法（算法）

```
# 循环，直到用户完成查询信息输入
while (用户未完成查询信息输入):
    # 获取用户输入的课程名称和课程编号
    课程名称 = 输入课程名称()
    课程编号 = 输入课程编号()
    # 检查输入信息是否为空，若为空则提示重新输入
    if (课程名称 为空 and 课程编号 为空):
        显示请至少输入课程名称或课程编号其中一项提示()
    else:
        break  # 跳出循环，认为用户完成输入

# 连接数据库
connect_database()

# 构建 SQL 查询语句
QUERY = "SELECT * FROM courses WHERE 1 = 1"
if (课程名称 不为空):
    QUERY = QUERY + " AND course_name LIKE '%" + 课程名称 + "%'"
if (课程编号 不为空):
    QUERY = QUERY + " AND course_id = '" + 课程编号 + "'"

# 执行 SQL 查询语句
```

```
执行(QUERY)

# 获取查询结果
results = FETCH_QUERY_RESULTS

if (results 不为空):
    # 处理并显示查询结果
    for result in results:
        显示(result.course_name + " - 学分: " + str(result.credit) + " - 课时: " +
            str(result.class_hours) + " - 授课教师: " + result.teacher)
else:
    # 显示未查询到课程提示
    显示("无匹配课程")

# 关闭数据库连接
close_database_connection()
```

4.2.7.7 流程图

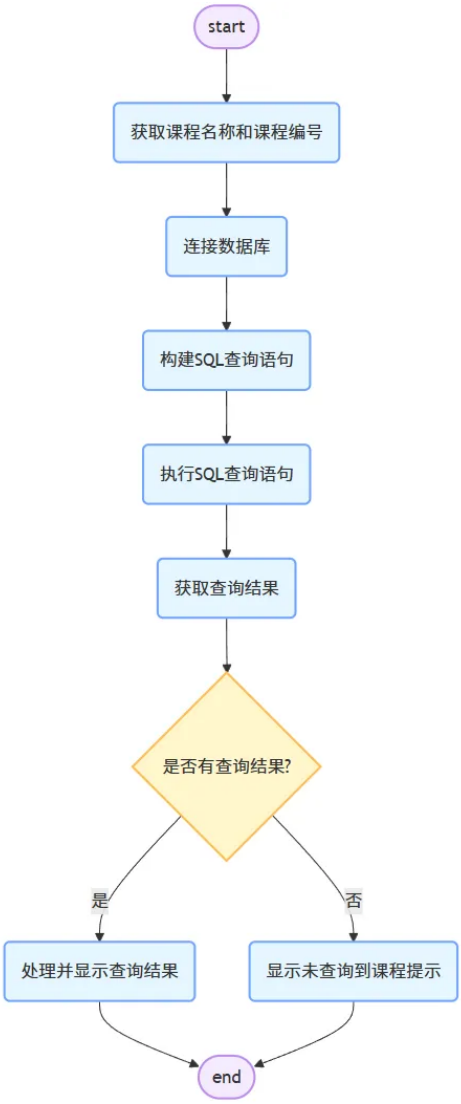


图 48: 查询课程信息流程图

4.2.8 教师上传成绩

4.2.8.1 模块概述

该模块主要为教师提供上传学生课程成绩信息的功能。教师通过系统操作界面，选择对应的课程以及学生，输入相应的成绩数据。系统连接数据库，将教师上传的成绩信息准确无误地添加到数据库中的成绩表内，最后向教师反馈上传操作的结果，以保障学生成绩数据的及时录入和准确记录。

4.2.8.2 IPO 图

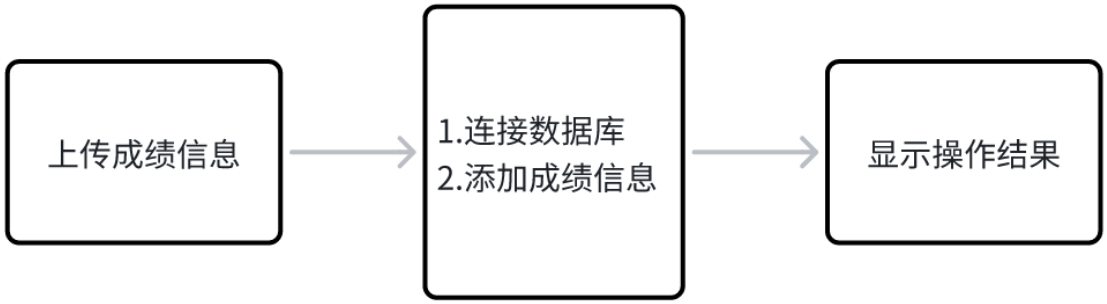


图 49: 教师上传成绩 IPO 图

4.2.8.3 功能

实现教师对学生课程成绩的上传功能，确保学生成绩能够准确、及时地录入教务管理系统，为学生成绩管理和后续教学评估等工作提供数据支持。

4.2.8.4 输入项

名称	标识	类型和格式	输入方式
课程编号	course_id	string	外部输入（教师在操作界面选择或输入）
学生学号	student_id	string	外部输入（教师在操作界面选择或输入）
成绩分数	score	float（假设成绩允许小数，可根据实际调整）	外部输入（教师在操作界面输入）

表 21: 教师上传成绩输入项

4.2.8.5 输出项

名称	标识	类型和格式	输出方式
操作结果	operation_result	boolean	由脚本输出
失败提示	failure_tip	string	由脚本输出（操作失败时提供）

表 22: 教师上传成绩输出项

4.2.8.6 设计方法（算法）

```
# 循环，直到教师完成成绩信息输入
while (教师未完成成绩信息输入):
    # 获取教师输入的课程编号、学生学号、成绩分数
    课程编号 = 输入课程编号()
    学生学号 = 输入学生学号()
    成绩分数 = 输入成绩分数()
    # 检查输入信息是否为空，若为空则提示重新输入
    if (课程编号 为空 or 学生学号 为空 or 成绩分数 为空):
        显示请完整输入课程编号、学生学号和成绩分数提示()
    else:
        break  # 跳出循环，认为教师完成输入

# 连接数据库
connect_database()

# 检查课程编号和学生学号是否在数据库中存在（假设二者都存在才能添加成绩）
执行查询语句 "SELECT * FROM courses WHERE course_id =?" WITH PARAMETERS (课程编号)
course_result = FETCH_QUERY_RESULT
执行查询语句 "SELECT * FROM students WHERE student_id =?" WITH PARAMETERS (学生学号
    )
student_result = FETCH_QUERY_RESULT

if (course_result 存在 and student_result 存在):
    # 构建插入语句，将成绩信息添加到数据库
    执行插入语句 "INSERT INTO scores (course_id, student_id, score) VALUES (?,?,?)"
        WITH PARAMETERS (课程编号, 学生学号, 成绩分数)
    commit_database_transaction()
    显示成绩上传成功提示()
    return 操作成功
else:
    显示操作失败，课程或学生信息不存在提示()
    return 操作失败

# 关闭数据库连接
close_database_connection()
```

4.2.8.7 流程图

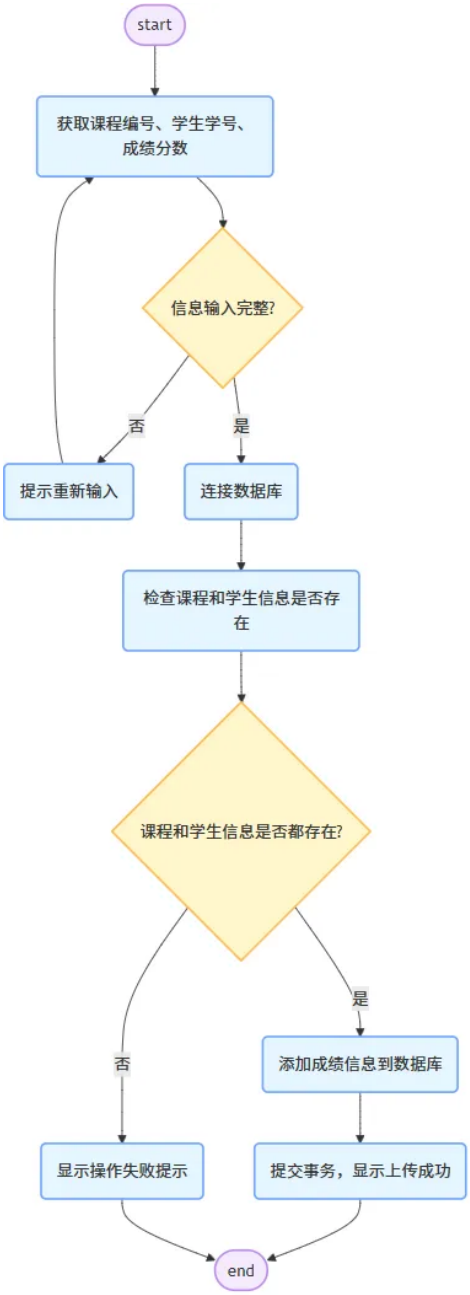


图 50: 教师上传成绩流程图

4.2.9 教师/学生查询成绩

4.2.9.1 模块概述

该模块为教师和学生提供查询成绩的功能。教师可通过此模块查询所授班级学生成绩情况，进行数据处理分析，如计算成绩分段等，以便了解教学效果；学生则能查询个人成绩。用户输入查询成绩指令及相关条件（如学生学号、班级编号等），系统连接数据库获取对应成绩信息，经处理后以可视化形式展示结果。

4.2.9.2 IPO 图

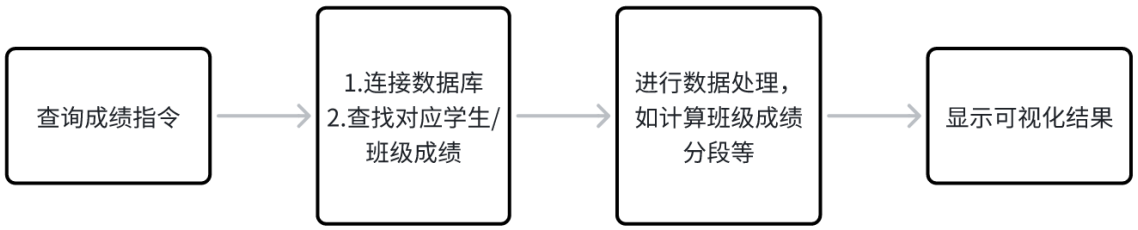


图 51: 教师/学生查询成绩 IPO 图

4.2.9.3 功能

实现教师和学生对成绩的查询及相关数据处理与可视化展示，为教学评估和学生了解自身学习情况提供便利。

4.2.9.4 输入项

名称	标识	类型和格式	输入方式
查询指令类型	query_type	string（取值为 “学生个人成绩”“班级成绩”等）	外部输入（用户在查询界面选择）
学生学号（若查询学生个人成绩）	student_id	string	外部输入（用户在查询界面输入）
班级编号（若查询班级成绩）	class_id	string	外部输入（用户在查询界面输入）

表 23: 教师/学生查询成绩输入项

4.2.9.5 输出项

名称	标识	类型和格式	输出方式
可视化结果	visualization_result	结构化数据（包含图表数据、统计数值等，依可视化展示需求）	由脚本输出并展示在查询结果页面
提示信息	tip_info	string （未查询到结果时为 “无匹配成绩” 等提示）	由脚本输出并展示在查询结果页面

表 24: 教师/学生查询成绩输出项

4.2.9.6 设计方法（算法）

```
# 循环，直到用户完成查询信息输入
while (用户未完成查询信息输入):
    # 获取用户输入的查询指令类型
    查询指令类型 = 输入查询指令类型()
    if (查询指令类型 为空):
        显示请输入有效的查询指令类型提示()
    elif (查询指令类型 not in ["学生个人成绩", "班级成绩"]):
        显示无效的查询指令类型，请输入学生个人成绩或班级成绩提示()
    else:
        if (查询指令类型 == "学生个人成绩"):
            # 获取学生学号
            学生学号 = 输入学生学号()
            if (学生学号 为空):
                显示请输入有效的学生学号提示()
            else:
                break  # 跳出循环，认为用户完成输入
        elif (查询指令类型 == "班级成绩"):
            # 获取班级编号
            班级编号 = 输入班级编号()
            if (班级编号 为空):
                显示请输入有效的班级编号提示()
            else:
                break  # 跳出循环，认为用户完成输入

# 连接数据库
connect_database()

# 根据查询指令类型构建不同 SQL 查询语句
if (查询指令类型 == "学生个人成绩"):
    QUERY = "SELECT * FROM scores WHERE student_id =?"
    PARAMETERS = (学生学号,)
elif (查询指令类型 == "班级成绩"):
    QUERY = "SELECT * FROM scores WHERE class_id =?"
    PARAMETERS = (班级编号,)

# 执行 SQL 查询语句
执行查询语句 QUERY WITH PARAMETERS PARAMETERS
results = FETCH_QUERY_RESULTS

if (results 不为空):
    if (查询指令类型 == "班级成绩"):
        # 计算成绩分段（假设成绩分段为 0 - 59, 60 - 69, 70 - 79, 80 - 89, 90 - 100
            ）
        low_count = 0
        medium_low_count = 0
        medium_count = 0
        medium_high_count = 0
        high_count = 0
```

```
        for result in results:
            score = result.score
            if (score >= 0 and score <= 59):
                low_count += 1
            elif (score >= 60 and score <= 69):
                medium_low_count += 1
            elif (score >= 70 and score <= 79):
                medium_count += 1
            elif (score >= 80 and score <= 89):
                medium_high_count += 1
            elif (score >= 90 and score <= 100):
                high_count += 1
        # 可继续计算平均分等其他统计数据
        # 构建可视化结果数据结构
        visualization_result = {
            "low_count": low_count,
            "medium_low_count": medium_low_count,
            "medium_count": medium_count,
            "medium_high_count": medium_high_count,
            "high_count": high_count
        }
    elif (查询指令类型 == "学生个人成绩"):
        # 直接将查询结果构建为可视化结果数据结构
        visualization_result = results[0]
    # 显示可视化结果
    显示(visualization_result)
else:
    # 显示未查询到成绩提示
    显示("无匹配成绩")

# 关闭数据库连接
close_database_connection()
```

4.2.9.7 流程图

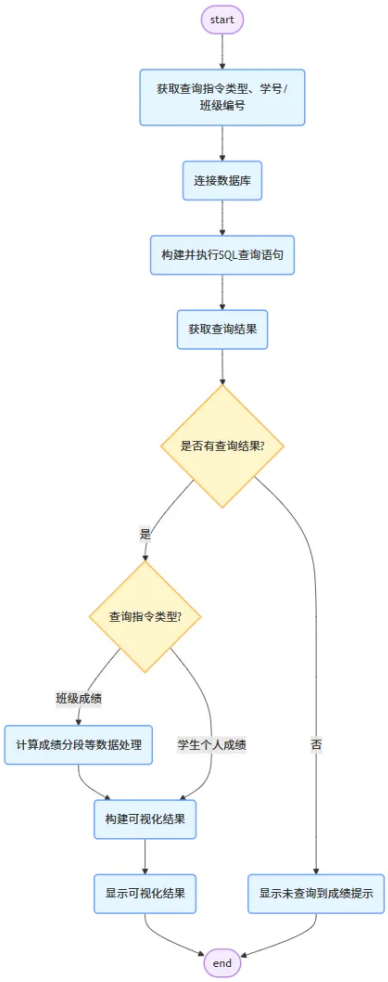


图 52: 教师/学生查询成绩流程图

4.2.10 修改成绩

4.2.10.1 模块概述

该模块用于处理成绩修改相关操作。教师或学生可提交成绩修改申请及修改信息，管理员对申请进行审核。若审核通过，系统连接数据库完成成绩修改；若未通过，则不做修改。此模块保证成绩修改操作的规范性和准确性，防止随意篡改成绩。

4.2.10.2 IPO 图

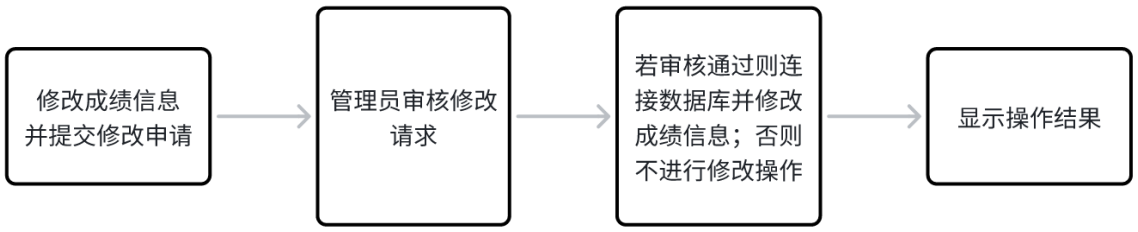


图 53: 修改成绩 IPO 图

4.2.10.3 功能

实现成绩修改申请的提交、审核及修改操作，保障成绩数据的准确性和安全性，为成绩管理提供合理的变更途径。

4.2.10.4 输入项

名称	标识	类型和格式	输入方式
课程编号	course_id	string	外部输入（用户在申请界面输入）
学生学号	student_id	string	外部输入（用户在申请界面输入）
修改后的成绩	new_score	float	外部输入（用户在申请界面输入）
修改原因	reason_for_modification	string	外部输入（用户在申请界面输入）

表 25: 修改成绩输入项

4.2.10.5 输出项

名称	标识	类型和格式	输出方式
操作结果	operation_result	string （如 “审核通过，成绩修改成功”“审核不通过，原因：[具体原因]”）	由脚本输出并展示给用户

表 26: 修改成绩输出项

4.2.10.6 设计方法（算法）

```
# 循环，直到用户完成修改信息输入
while (用户未完成修改信息输入):
    # 获取用户输入的课程编号、学生学号、修改后的成绩、修改原因
    课程编号 = 输入课程编号()
    学生学号 = 输入学生学号()
    修改后的成绩 = 输入修改后的成绩()
    修改原因 = 输入修改原因()
    # 检查输入信息是否为空，若为空则提示重新输入
    if (课程编号 为空 or 学生学号 为空 or 修改后的成绩 为空 or 修改原因 为空):
        显示请完整输入课程编号、学生学号、修改后的成绩和修改原因提示()
    else:
        break  # 跳出循环，认为用户完成输入

# 提交修改申请，等待管理员审核
提交修改申请(课程编号，学生学号，修改后的成绩，修改原因)

# 等待管理员审核结果
while (管理员未审核):
    pass  # 等待

# 获取管理员审核结果
审核结果 = 获取审核结果()

if (审核结果 == "通过"):
    # 连接数据库
    connect_database()
    # 构建SQL更新语句
    UPDATE_QUERY = "UPDATE scores SET score =? WHERE course_id =? AND student_id =?
        "
    PARAMETERS = (修改后的成绩，课程编号，学生学号)
    # 执行SQL更新语句
    执行(UPDATE_QUERY，PARAMETERS)
    commit_database_transaction()
    显示("审核通过，成绩修改成功")
else:
    显示("审核不通过，原因：" + 获取审核不通过原因())

# 关闭数据库连接
close_database_connection()
```

4.2.10.7 流程图

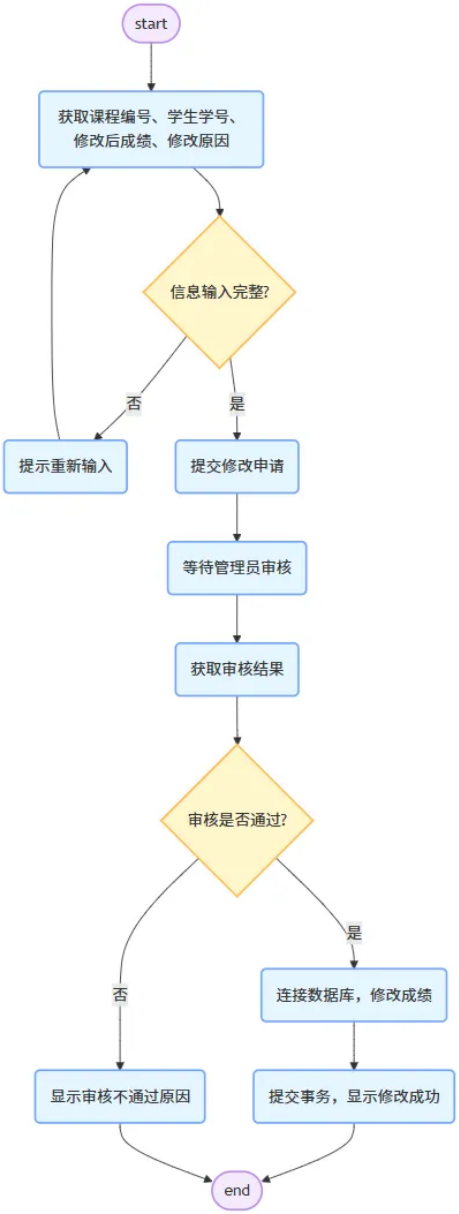

start
获取课程编号、学生学号、
修改后成绩、修改原因
信息输入完整?
否
是
提示重新输入
提交修改申请
等待管理员审核
获取审核结果
审核是否通过?
否
是
显示审核不通过原因
连接数据库，修改成绩
提交事务，显示修改成功
end

图 54: 修改成绩流程图

5 用户界面

5.1 用户登录界面



图 55: 登录界面

如上图为用户登录界面，未注册的用户点击下方蓝字部分即可注册，已有帐户的用户输入相应用户名与密码即可登录。可以使用系统注册的身份登录，也可直接使用统一身份进行认证登录。

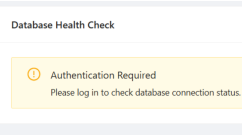


图 56: 身份认证界面

用户会看到如上界面，需要进行身份认证登录才可以进入确认数据库的连接状态。

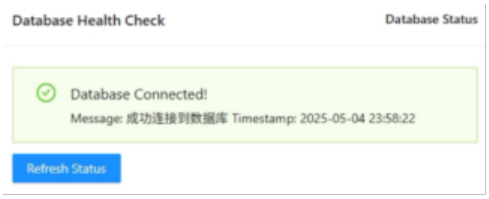


图 57: 认证成功界面

身份认证通过后，即可查看数据库连接状态，可以在主页看到如上图提示。

5.2 系统主页界面

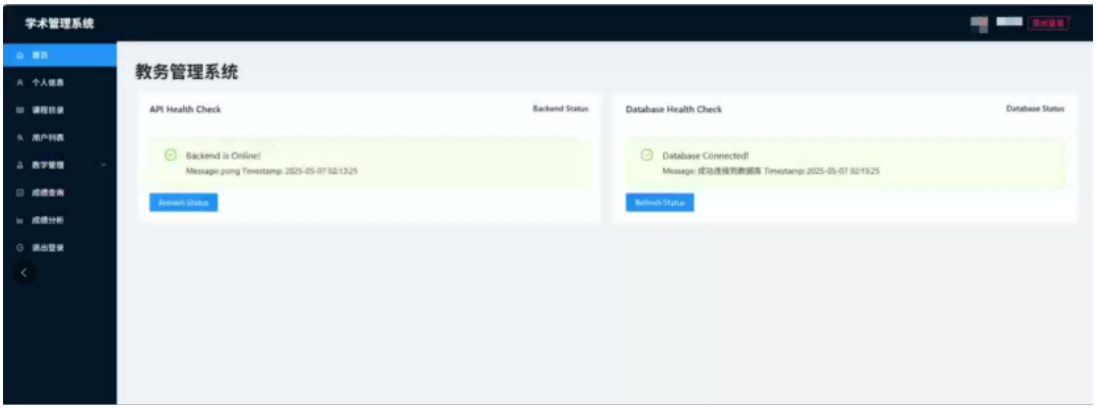


图 58: 系统主页界面

登录后进入系统主页，可以看到如上界面。左侧栏有“个人信息”，“课程信息”，“用户列表”这几个选项，点击选择相应选项，可以进入相应界面实现相应目标。在主页点击“退出登录”可实现账户退出登录。

5.3 用户列表界面

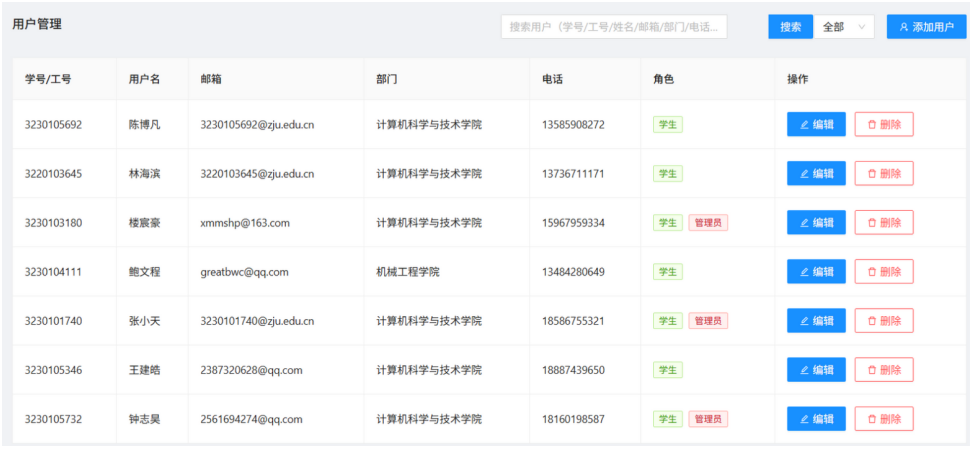


图 59: 用户列表界面

如上图为管理员可获得的用户列表图，可以看到用户的学号、用户名、邮箱地址、所属院系、联系方式等信息。可以看到“角色”这一栏标注出了用户的角色身份，对应有学生、管理员、教师几种身份。管理员可以通过右侧“编辑”、“删除”等选项对用户信息进行修改或注销。如需查询用户，可以直接在上方搜索框进行快捷筛选和查询。

5.4 课程列表界面



图 60: 课程列表界面

如上图为管理员可获得的课程列表图，可以看到各个课程的课程编号、课程名称、课程代码、学分、所属院系等信息。如需查询相应课程，可通过上方搜索框进行快捷筛选和查询。

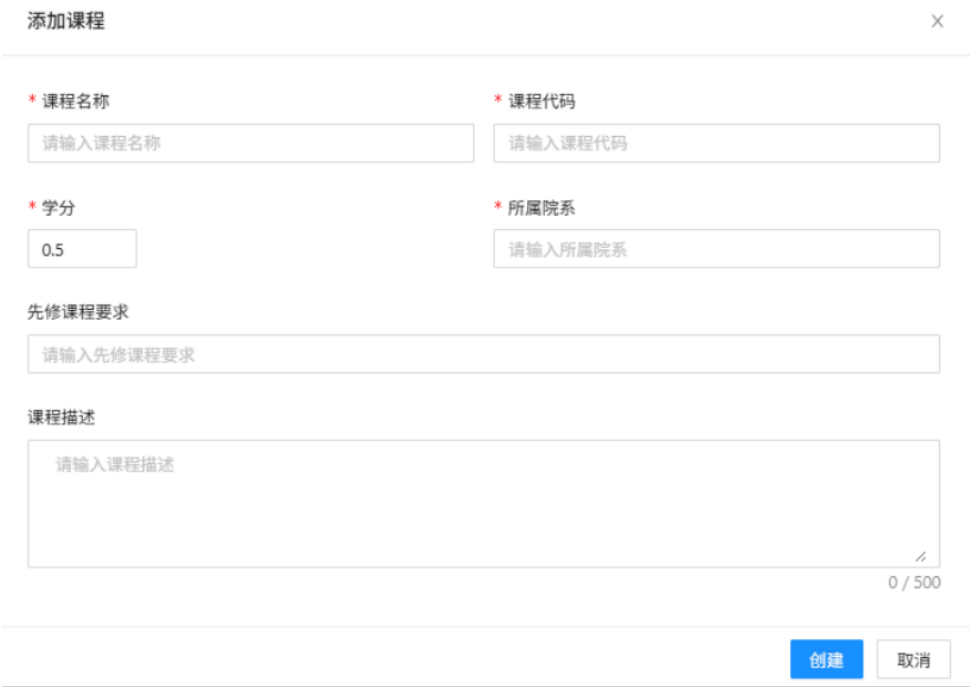


图 61: 修改课程信息界面

管理员可通过右侧“编辑”、“删除”等选项对课程信息进行修改，如上图，按照界面提示输入相应信息。

5.5 课程目录界面

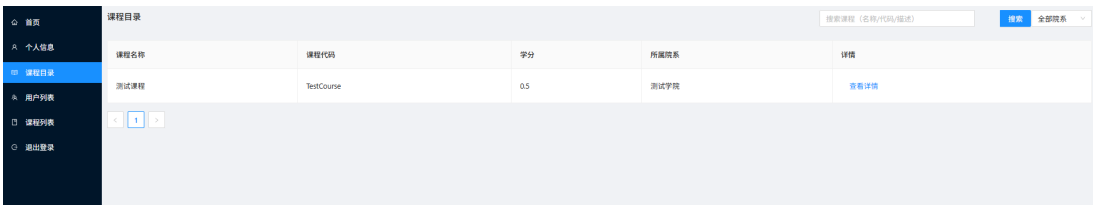


图 62: 课程目录界面

如上图为课程目录界面，和课程列表界面的区别是该界面只能进行课程信息的查询而不支持课程信息的修改等操作。同样可以通过上方搜索框进行快捷筛选和查询，还可以点击右侧蓝字查看课程详情，获得课程预修要求、课程描述等详细信息。

5.6 成绩查询界面



图 63: 成绩查询界面

如上图为学生成绩查询界面，学生可以选择相应学年相应学期，从而查询到学期均绩、平均分、总学分和所选课程等信息。在左侧课程成绩列表点击选择相应课程可查看该课程更细致的成绩分析。



图 64: 课程成绩分析界面 1

如上，系统可根据该课程获得分数的加权组成为用户生成对应雷达图，使得用户可以更加直观地对比各项成绩。



图 65: 课程成绩分析界面 2

如上。系统可生成表格使用户更清晰地获得该课程各项评分标准的原始成绩、成绩占比以及加权得分等信息。

5.7 成绩分析界面



图 66: 成绩分析界面

如上图为系统成绩分析界面，用户可通过上方搜索框筛选相应学年相应学期相应课程，可获得该课程总成绩的平均分、最高分、最低分、分数标准差等信息，从而对自己的学习成果进行评估。

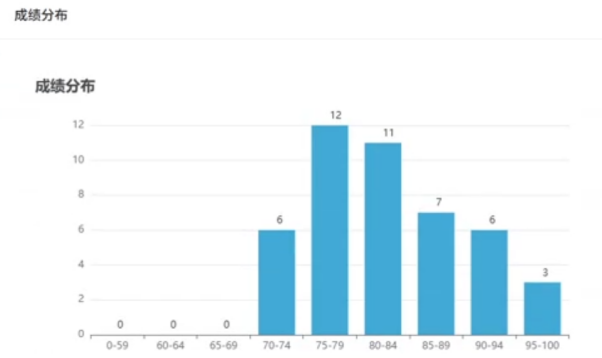


图 67: 成绩分布界面



图 68: 等级分布界面

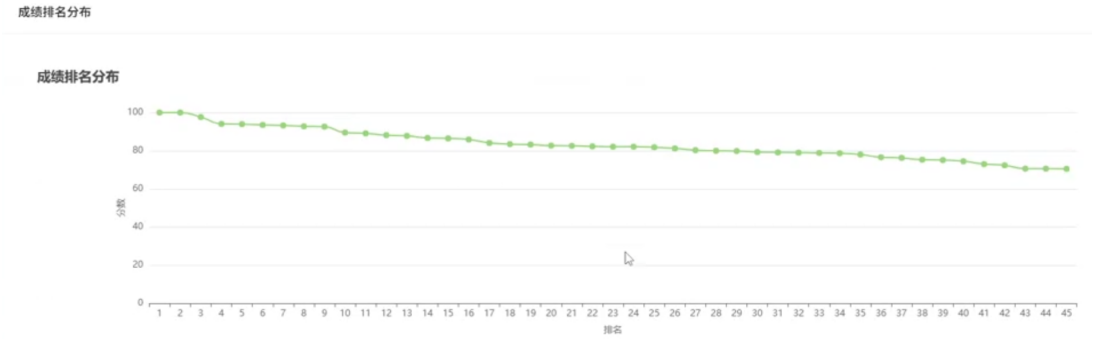


图 69: 成绩排名分布界面

学生成绩列表

学号	姓名	成绩	等级	排名	百分位
S2023006	学生7	100	A	1	100.0%
S2023020	学生21	100	A	2	97.8%
S2023034	学生35	97.65851367541024	A	3	95.6%
S2023015	学生16	94.06711178722432	A	4	93.3%
S2023044	学生45	93.94565358983424	A	5	91.1%
S2023001	学生2	93.4917846124600	A	6	88.9%
S2023017	学生18	93.24120447650784	A	7	86.7%
S2023022	学生23	92.77325831855048	A	8	84.4%
S2023008	学生9	92.60488228399992	A	9	82.2%
S2023011	学生12	89.494850417864424	A-	10	80.0%

图 70: 成绩列表界面

如上面界面所示，系统会生成相应柱状图、饼状图、折线图、表格等对成绩排名、分数分布、等级分布等数据进行更直观地表达，使用户更好地对自己学习成果进行自我评估。

6 数据库设计

6.1 概念结构设计

6.1.1 用户信息

User 为强实体, 如图:

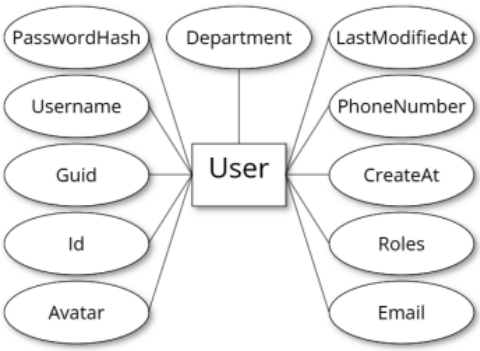


图 71: 用户信息 E-R 图

6.1.2 课程信息

Course 为强实体, 如图:

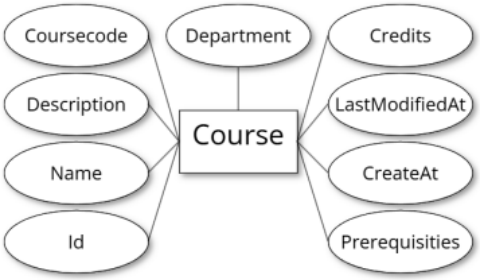


图 72: 课程信息 E-R 图

6.1.3 选修信息

Enrollment 为强实体, 如图:

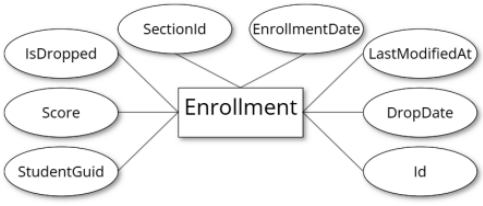


图 73: 选修信息 E-R 图

6.1.4 班级信息

Section 为强实体, 如图:

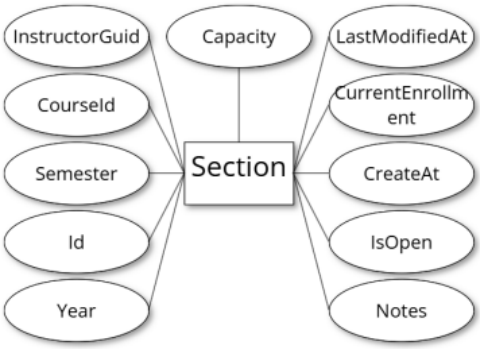


图 74: 班级信息 E-R 图

6.1.5 课表信息

ClassSchedule 为强实体, 如图:

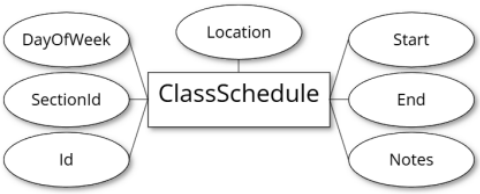


图 75: 课表信息 E-R 图

6.1.6 总 E-R 图

信息管理子系统模块 E-R 图如下：

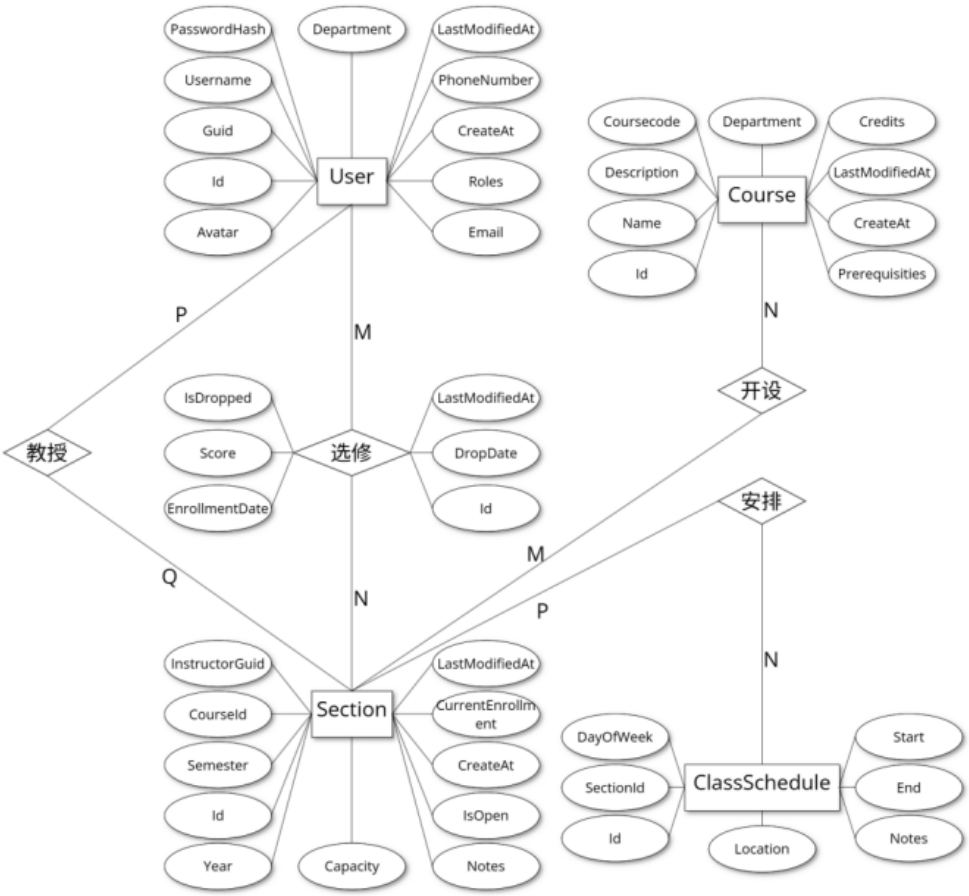


图 76: 模块 E-R 图

教学服务系统总 ER 图如下：

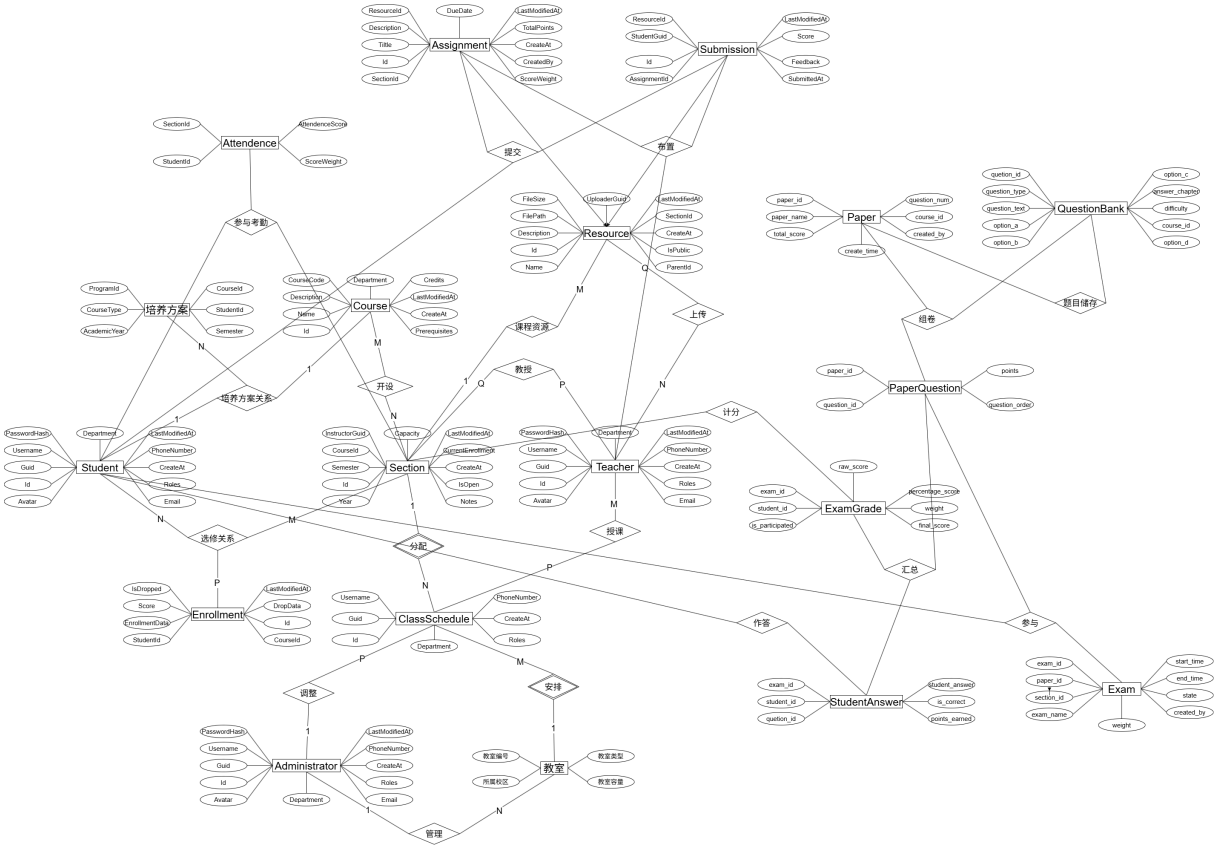


图 77: 系统总 E-R 图

6.2 逻辑结构设计

用户

User(Id,Username,PasswordHash,Roles,***Guid***,Avatar,CreateAt,Department,Email,LastModifiedAt, PhoneNumber)

课程

Course(***Id***,Name,Description,Credits,CourseCode,Department,Prerequisites,CreateAt, LastModifiedAt)

选修

Enrollment(***Id***,StudentGuid,SectionId,Score,IsDropped,EnrollmentDate,DropDate, LastModifiedAt)

班级

Section(***Id***,Year,Semester,CourseId,InstructorGuid,Capacity,CurrentEnrollment,IsOpen,Notes,CreateAt, LastModifiedAt)

课表

ClassSchedule(***Id***,SectionId,DayOfWeek,Start,End,Location,Notes)

6.3 物理结构设计

本系统的数据库表如下：

编号	属性名	字段名	数据类型	备注
1	用户编号	Id	Text	
2	用户名字	Username	Text	
3	密码	PasswordHash	Text	
4	用户角色	Roles	Integer[]	
5	唯一标识符	Guid	Uuid	主键
6	用户资料	Avatar	Bytea	
7	创建时间	CreateAt	Timestamp with time zone	
8	所属院系	Department	Text	
9	用户邮箱	Email	Text	
10	最后修改时间	LastModifiedAt	Timestamp with time zone	
11	用户手机号	PhoneNumber	Text	

表 27: 用户信息表

编号	属性名	字段名	数据类型	备注
1	课程编号	Id	Integer	主键
2	课程名称	Name	Text	
3	课程描述	Description	Text	
4	课程学分	Credits	Numeric	
5	课程代码	CourseCode	Text	
6	所属院系	Department	Text	
7	预修要求	Prerequisites	Text	
8	创建时间	CreateAt	Timestamp with time zone	
9	最后修改时间	LastModifiedAt	Timestamp with time zone	

表 28: 课程信息表

编号	属性名	字段名	数据类型	备注
1	选修编号	Id	Integer	主键
2	学生标识符	StudentGuid	Uuid	外键
3	班级编号	SectionId	Integer	外键
4	成绩	Score	Numeric	
5	是否弃修	IsDropped	Boolean	
6	选修时间	EnrollmentDate	Timestamp with time zone	
7	弃修时间	DropDate	Timestamp with time zone	
8	最后修改时间	LastModifiedAt	Timestamp with time zone	

表 29: 选修信息表

编号	属性名	字段名	数据类型	备注
1	班级编号	Id	Integer	主键
2	年份信息	Year	Integer	
3	学期信息	Semester	Integer[]	
4	课程编号	CourseId	Integer	外键
5	教师标识符	InstructorGuid	Uuid	外键
6	课程容量	Capacity	Integer	
7	已选人数	CurrentEnrollment	Integer	
8	是否开课	IsOpen	Boolean	
9	备注信息	Notes	Text	
10	创建时间	CreateAt	Timestamp with time zone	
11	最后修改时间	LastModifiedAt	Timestamp with time zone	

表 30: 班级信息表

编号	属性名	字段名	数据类型	备注
1	排课编号	Id	Integer	主键
2	班级编号	SectionId	Integer	外键
3	每周时间	DayOfWeek	Integer	
4	开始时间	Start	Integer	
5	结束时间	End	Integer	
6	上课地点	Location	Text	
7	备注信息	Notes	Text	

表 31: 排课信息表

7 运行设计

7.1 运行模块组合

本子系统按照角色划分为用户模块和管理员模块，每个模块根据功能进一步细分为课程管理、成绩管理、权限控制等界面。模块之间共享基础组件，例如登录界面、导航栏和公共 API 接口。具体模块组合如下：

- **用户模块**：包含学生和教师角色，功能包括课程查询、成绩查看、个人信息修改、选课申请等。
- **管理员模块**：包含教务管理人员角色，功能包括账户管理、课程信息维护、权限分配、日志审计等。

模块间通过统一的 RESTful API 进行数据交互，前端采用 Blazor 框架实现动态页面渲染，后端通过 ASP.NET Core 处理业务逻辑，数据库使用 PostgreSQL 存储结构化数据。

7.2 运行控制

用户或管理员需通过统一登录入口选择身份（学生/教师/管理员），输入账号密码完成认证。系统根据角色权限跳转至对应功能界面。各模块操作流程如下：

7.2.1 用户模块

1. **课程查询**：用户输入课程名称或编号，系统返回匹配结果及详细信息。
2. **个人信息修改**：用户修改个人信息并提交，系统返回修改结果。
3. **成绩查看**：学生可查看本人成绩；教师可查看所授课程的学生成绩列表。
4. **选课申请**：学生提交选课请求，系统验证课程容量及时间冲突后更新数据库。

7.2.2 管理员模块

1. **账户管理**：管理员可添加/删除账户、重置密码、分配角色权限。
2. **课程排课**：设置课程时间、地点、容量，系统自动检测教室冲突。
3. **数据导出**：支持将成绩表导出为 Excel 文件，系统校验文件格式后生成下载链接。

所有操作均通过界面按钮触发，后端接口验证用户权限后执行数据库操作，并返回操作结果提示。

8 系统出错设计

8.1 出错信息

系统通过统一错误码和友好提示信息反馈异常情况，关键错误类型及处理方式如下：

错误类型	错误信息形式	可能原因	处理方法
数据库连接失败	弹窗提示“系统繁忙，请稍后重试”	1. 数据库配置错误 2. 连接池耗尽	1. 检查配置文件 2. 重启服务
权限不足	页面提示“您无权执行此操作”	用户越权访问管理功能	记录日志并跳转至默认页面
成绩录入冲突	弹窗提示“该学生成绩已存在”	重复提交同一学生成绩	提示覆盖或取消操作
JWT 无效	页面提示“登录信息无效”	长时间未操作导致 JWT 过期	自动登出并返回登录界面
文件格式错误	提示“仅支持.xlsx 格式”	上传非 Excel 文件	强制重新选择文件

表 32: 系统错误处理表

8.2 补救措施

8.2.1 数据恢复机制

- **自动备份**：每日凌晨 3 点全量备份数据库，保留最近 7 天备份文件。
- **增量备份**：每 30 分钟记录事务日志，支持恢复到任意时间点。
- **手动恢复**：管理员可通过 pgAdmin 工具选择备份文件执行恢复操作。

8.2.2 故障转移策略

1. 部署双节点 PostgreSQL 集群，主节点故障时自动切换至备用节点。
2. 前端配置负载均衡，单台服务器宕机不影响整体服务可用性。

8.2.3 日志审计

- 记录所有敏感操作（如成绩修改、权限变更），包含操作者 IP、时间戳及详细参数。
- 日志文件加密存储，保留周期为 2 年，支持按关键词检索和导出审计报告。

8.3 系统维护设计

1. **安全更新**：每月检查.NET Core 和 PostgreSQL 安全补丁，24 小时内完成高危漏洞修复。
2. **性能监控**：使用 Prometheus 监控 API 响应时间，超过 2 秒的请求触发告警。
3. **输入过滤**：对学号、课程编号等字段实施正则校验（如\d{10}），拦截非法字符。
4. **会话管理**：用户 15 分钟无操作后自动登出，JWT 令牌有效期设置为 1 小时。
5. **CSRF 防护**：所有表单提交需携带动态 Token，后端验证 Token 有效性。
6. **XSS 防护**：对用户输入的文本内容进行 HTML 转义，禁止内联脚本执行。